North SDK User Manual

C++ · Python

Contents

SDK Overview

- Introduction
- Modules
- Interaction patterns
- Runtime environment
- Reading path

Application Development

- Packages and runtime environment
- Quick Start
  - 1. Connect the robot
  - 2. Set the SDK and robot address
  - 3. Run the read-only diagnostic
  - 4. Receive left-arm state
  - Python users
  - Build the C++ example (optional)
  - Completion criteria
  - First-run troubleshooting
- Time synchronization
- Video environment setup
  - NVIDIA drivers and bundled runtime
  - Check the video environment
  - Receive the first video stream
- Subscriptions and caches
- Service results
- Motion control
  - Prepare and end a task
  - Joint arrays and coordinates
- Motion protection and trajectory processing
  - Command admission and fault protection
  - Position ranges and velocity constraints
  - Smoothing continuous position commands
- Shutdown and reconnect
- Troubleshooting

Software Service Interface

- API conventions

- Common parameters and results
- SdkConfig defaults
- Errors and result checking
- Python mapping
  - Common Python signatures
- Public capability scope
- Client APIs
  - NorthSDK
  - Subscription
  - ConfigClient
  - MasterClient
  - WholeBodyClient
  - ChassisClient
  - TorsoClient
  - ArmClient
  - HandClient
  - SensorClient
  - PowerClient
  - VideoClient
  - AlignmentClient
- Time-sync parameters
- Data types and fields
  - Common field meanings
  - Configuration data
  - Video and alignment data
  - Basic type declarations
- Lifecycle and diagnostics

Example Code Reference

- Example index
- Included quickstart programs
- Read-only connection diagnostics
- Shared entry point
- Subscribe to Master status
  - C++
  - Python
- Read cached left-arm state
  - C++
  - Python
- Subscribe to torso state
  - C++

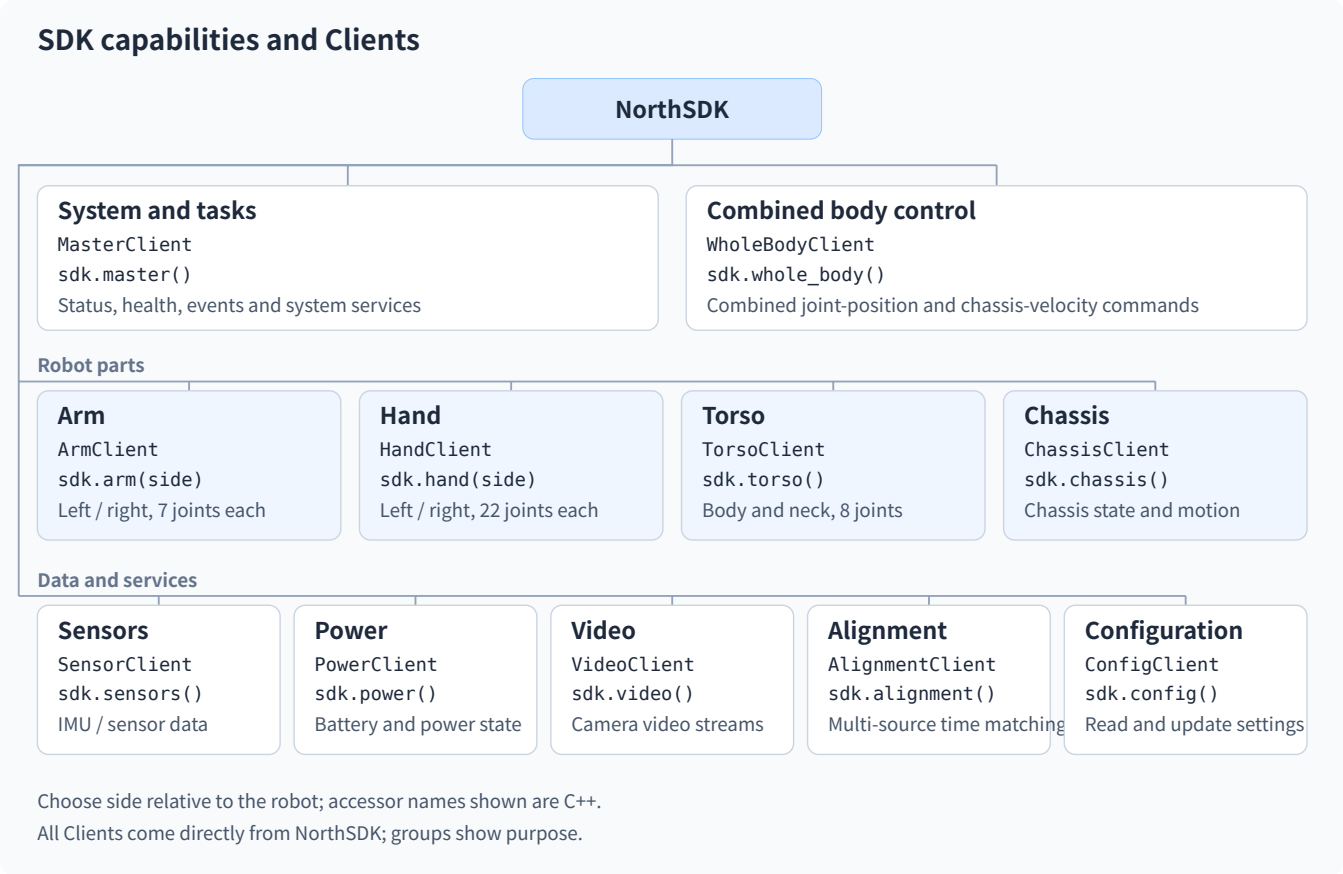
Python
Subscribe to hand tactile images
C++
Python
Query hand calibration status
C++
Python
Subscribe to IMU
C++
Python
Subscribe to Lidar points
C++
Python
Read valid battery measurements
C++
Python
Subscribe to head JPEG
Browser preview
C++
Python
Align two-arm state
C++
Python
Query configuration instances
C++
Python
Master mode service
C++
Python
Send a single-arm target
C++
Python
WholeBody two-arm command
C++
Python

SDK Overview

Introduction

North SDK provides C++ and Python APIs for Sharpa North robots. Applications use Clients to receive robot state, call device services and send joint-position and chassis-velocity commands.

C++ APIs are in namespace `north`, with public header `<north_sdk/NorthSDK.h>`. The Python package is `north_sdk`. Both languages share the same capabilities, with language-specific results and resource management.



The figure uses C++ accessors. Side refers to the robot's own left or right; Python uses Side.LEFT / Side.RIGHT.

`NorthSDK` is the common entry point; accessors return each Client directly. `MasterClient` manages system state and tasks, part Clients control their mechanisms, and `WholeBodyClient` combines commands for multiple parts. Other Clients provide sensing, video, alignment, power and configuration. The figure groups Clients by purpose.

Modules

Module	Purpose
Master	Runtime status, health, task events and system management
WholeBody	Combine joint-position targets and chassis velocity commands
Chassis	State, velocity commands, mode switching, enable and reset
Torso	Eight torso/neck joints: state, position commands and services
Arm	Seven joints per arm: state, position commands and services
Hand	Twenty-two joints per hand: state, position commands, tactile data and services
Sensors	IMU, ultrasonic, tactile skin and Lidar
Power	Power-board and battery state, and power management
Video	Decode head stereo H.265 and wrist H.264 streams and deliver JPEG callbacks; NVIDIA NVDEC is recommended, with software decoding as a compatibility option
Alignment	Match video, device state, and Master state by source timestamp
Config	Discover and safely modify robot-exposed persistent user configuration

Available capabilities depend on robot configuration and software version. See the API reference in Software Service Interface.

Interaction patterns

Pattern	Typical API	Meaning
Subscription	`on_*()`	Receive state/events continuously and retain the subscription handle
Cached state	`latest_*(max_age)`	Read the latest local sample subject to a freshness limit
Command	`send_*()`	Submit a command, then observe device feedback
Service	`set_mode()` / `reset()`	Wait for a response and check SDK errors and business results

Runtime environment

Item	Requirement
Runtime system	Recommended: Ubuntu 22.04 LTS x86_64 / Ubuntu 24.04 LTS x86_64

Item	Requirement
C++ API	C++17, using public headers and libraries from the delivered package
Python API	Recommended: CPython 3.10.x / 3.12.x; the interpreter minor version and architecture must match a binding included in the package. See the package manifest
Robot	Matching software, a reachable communication address and required devices
Video component	All packages include video support, GStreamer runtime libraries and required plugins. Default hardware decoding requires a discrete NVIDIA GPU supporting H.264/H.265 decoding and a compatible driver.

Available capabilities depend on the delivered package, runtime dependencies, and robot configuration. Use C++ headers, Python extensions and SDK libraries from one delivery package. Refer to the delivery notes for matching robot software versions.

Time synchronization is enabled by default. Complete the one-time host setup before first use. The default software mode adjusts host system time; see Time synchronization.

Reading path

1. Application Development: complete read-only bring-up in Quick Start, then learn about resources, service results and motion admission.
2. Example Code Reference: choose C++ or Python examples by task.
3. Software Service Interface: look up support status, parameters, results, units and data fields.

Application Development

Packages and runtime environment

The package contains C++ headers and shared libraries, Python bindings, examples, and video runtime dependencies. This manual recommends the following development environment.

Development system	Recommended Python interpreter	C++ API
Ubuntu 22.04 LTS x86_64	CPython 3.10.x	C++17
Ubuntu 24.04 LTS x86_64	CPython 3.12.x	C++17

Other Linux x86_64 environments must meet the minimum runtime symbol requirements recorded in the package BUILD-MANIFEST.json. Compatibility requires validation in the target environment.

The development environment provides the Python interpreter. Its minor version and architecture must match a binding included in the package, as listed in the package manifest. This also applies to virtual environments. Python is unnecessary for C++-only applications.

Extract the entire package into a dedicated directory and preserve its structure. GStreamer runtime libraries and required video plugins are bundled; installing system GStreamer is unnecessary. NVIDIA drivers are not bundled; see Video environment setup.

Directory or file	Purpose
`include/north_sdk/`	Public C++ headers
`lib/`	SDK and supporting runtime libraries
`lib/python/`	Bindings for the target Python version
`lib/video-runtime/`	Bundled video libraries, plugins and tools
`bin/`	C++ bring-up and connection diagnostic programs
`examples/python/`	Python examples and environment check script
`manual/`	User manual
`setup.bash`	Load this package's library, Python and video plugin paths
`BUILD-MANIFEST.json`	Build information, Python ABI and dependencies

To update, replace the complete package, reload its environment and recheck import, read-only connection and state reception.

Quick Start

This section connects to the robot and receives the first state stream with the fewest steps, without sending motion, mode, calibration or power commands.

> **Safety:** On a first connection, run only the read-only programs in this section. Do not call `send_*()`, `set_mode()`, `enable()`, `reset()`, calibration or Power management APIs.

1. Connect the robot

Connect the development computer to the Ethernet port led out from the switch on the robot's rear right side. Power on the robot and wait for its software to start. State communication and time synchronization both use this wired connection.

The default robot IP is `192.168.100.100`; the SDK communicates over UDP port `7449`. For a direct connection, open Ubuntu Settings → Network → Wired → IPv4 and select Manual. For example, set the computer to `192.168.100.10` with netmask `255.255.255.0`, leaving gateway and DNS blank. Check that the address is unused. Use the actual subnet if changed at delivery.

2. Set the SDK and robot address

Replace the SDK path below with your extraction directory. Use the default robot address unless changed for your delivery.

```
export NORTHSDK_PREFIX=/absolute/path/to/north-sdk
export NORTHSDK_ENDPOINT=udp/192.168.100.100:7449
source "$NORTHSDK_PREFIX/setup.bash"
```

Environment variables apply only to this terminal and its child processes; configure new terminals separately.

`NORTHSDK_PREFIX` points to the extracted SDK root containing `setup.bash`.

Before first use of automatic time synchronization, run this one-time host setup from the ordinary development account:

```
sudo "$NORTHSDK_PREFIX/tools/setup-host" --user "$USER"
"$NORTHSDK_PREFIX/tools/setup-host" check
```

This installs the host time-sync component and authorizes the current account. Applications subsequently run without `sudo`; repeat setup from the new package after an SDK update. Software synchronization may step system time and temporarily pause system time services. Their previous service state is restored after the last time-sync session closes. See Time synchronization to disable it.

3. Run the read-only diagnostic

```
"$NORTHSDK_PREFIX/bin/north_sdk_robot_smoke" \
  --endpoint "$NORTHSDK_ENDPOINT" --seconds 5
```

When the basic connection is healthy, the final line is:

```
PASS
```

`PASS` means Master status, hardware health and at least one heartbeat were received. Check each required device stream against the actual configuration; `silent` means no sample was received for that stream.

4. Receive left-arm state

```
"$NORTHSDK_PREFIX/bin/north_sdk_quickstart" "$NORTHSDK_ENDPOINT"
```

When the left arm is online and publishing, output resembles:

```
left arm seq=1234 joints=7
```

Press Ctrl+C to exit. If the robot has no left arm, use streams for devices actually installed as reported by `robot_smoke`.

Python users

Run `python3 --version` and check that the interpreter minor version matches an included binding. CPython 3.10.x on Ubuntu 22.04 and CPython 3.12.x on Ubuntu 24.04 are recommended; virtual environments must also use an interpreter minor version supported by the package.

Step 2 already sets the Python module path. Run the example directly:

```
python3 "$NORTHSDK_PREFIX/examples/python/quickstart.py" \
  --endpoint "$NORTHSDK_ENDPOINT"
```

Build the C++ example (optional)

After the prebuilt program succeeds, verify the C++ development environment with one `g++` command:

```
g++ -std=c++17 "$NORTHSDK_PREFIX/examples/cpp/quickstart.cpp" \
  -I"$NORTHSDK_PREFIX/include" -L"$NORTHSDK_PREFIX/lib" -lnorth_sdk \
  -Wl,-rpath,"$NORTHSDK_PREFIX/lib" \
  -o north_sdk_quickstart

./north_sdk_quickstart "$NORTHSDK_ENDPOINT"
```

`-L` locates link libraries and `-Wl,-rpath` records the SDK library directory. Still load `setup.bash` before running to select bundled video libraries and plugins. Rebuild or update the runtime path if the SDK directory moves.

Completion criteria

The read-only bring-up is complete when:

- `robot_smoke` reports `PASS`;

- streams required by the application continuously receive samples;
- the C++ or Python example exits cleanly;

Continue with the subscription and motion guidance below, then choose task examples in Example Code Reference. If bring-up fails, see First-run troubleshooting below.

First-run troubleshooting

For first-connection, startup or Python import failures, check the package, shared libraries and network in that order. These checks reuse the environment from step 2 and are needed only on failure.

Package or version

```
test -f "$NORTHSDK_PREFIX/include/north_sdk/NorthSDK.h"
test -f "$NORTHSDK_PREFIX/lib/libnorth_sdk.so"
cat "$NORTHSDK_PREFIX/Version"
```

Python import check:

```
python3 -c "import north_sdk; print(north_sdk.NorthSDK.version())"
```

If you see `No module named 'north_sdk._north_sdk'`, check that the extension exists and matches your Python version:

```
python3 --version
ls "$NORTHSDK_PREFIX"/lib/python/north_sdk/_north_sdk*.so
```

Check the package manifest and extensions provided in `lib/python/north_sdk/`. A `cpython-312` filename tag requires CPython 3.12. Switch to a matching interpreter or obtain a package containing the required binding. Replace the package if an extension listed in its manifest is missing.

Shared libraries

```
ldd "$NORTHSDK_PREFIX/bin/north_sdk_quickstart" | \
  grep -E 'north_sdk|zenoh|protobuf|not found'
```

North SDK, Zenoh and Protobuf should come from one package. If a library is `not found` or resolves to another SDK/ROS environment, fix `LD_LIBRARY_PATH`.

Network

```
ip route get 192.168.100.100
```

Replace the IP with the actual robot address and verify the intended interface. Use `ping` only when ICMP is permitted. A failed ping does not necessarily mean Zenoh is unavailable; `robot_smoke` is authoritative.

Time synchronization

The SDK prepares gPTP synchronization after establishing communication. Complete the host setup before first use.

Option	Usage
Software mode (default)	Adjusts host system time. The NIC must support software transmit/receive timestamps.
Hardware mode	Before SDK creation, set `config.time_sync.mode` to `TimeSyncMode::kHardware` (C++) or `TimeSyncMode.HARDWARE` (Python). Requires hardware timestamps and a NIC PHC; adjusts only the NIC clock.
Disable	Set `config.time_sync.enabled = false` (C++) or `False` (Python) before SDK creation.

Check synchronization with:

```
python3 "$NORTHSDK_PREFIX/examples/python/time_sync_status.py" \
  --endpoint "$NORTHSDK_ENDPOINT" --mode software --seconds 10
```

When `state` is READY and `robot_time_ns` is present, use `sdk.robot_time_ns()` to read synchronized time. Robot time uses the gPTP domain and is not guaranteed to match UTC.

By default, a synchronization failure makes `connect()` return an error while retaining communication. Inspect `time_sync_diagnostics()` for the cause; the status script shows how to handle the error and continue using the connection. If synchronization is lost during operation, wait for readiness before resuming calculations that depend on synchronized time.

For communication-only checks, add `--no-time-sync` to `north_sdk_robot_smoke`. See Time-sync parameters for configuration and diagnostic fields.

Video environment setup

Complete this section when using video. First load the package environment as described in Quick Start. GStreamer runtime libraries, parsing/conversion/JPEG plugins, NVIDIA decoder plugins and a minimal H.264/H.265 software decoder are bundled; installing system GStreamer is unnecessary.

NVIDIA drivers and bundled runtime

NVIDIA NVDEC is the default backend. The discrete GPU must support the camera's H.264/H.265 coding profile, with NVIDIA driver and decode libraries matched to the GPU and Ubuntu release. Install NVIDIA drivers separately.

Check the driver first:

```
nvidia-smi -L
```

If no GPU is listed, follow the Ubuntu driver installation instructions to install or repair the driver. NVDEC uses driver-provided `libcuda.so.1` and `libnvcuvid.so.1`. See NVIDIA's decode support matrix for supported input profiles.

`setup.bash` sets SDK video library, plugin and cache paths. Load it in each terminal or application launch environment, and use the current SDK's runtime and plugins. If the application already loads another GStreamer runtime, receive video in a separate process.

The bundled software decoder supports compatibility and functional debugging. Select it explicitly when no NVIDIA GPU is available; it requires no NVIDIA driver. Software decoding has poor performance for multiple or high-resolution streams and may cause severe frame drops and latency, so it is not recommended for real-time tasks. The SDK never automatically falls back from hardware to software decoding.

Check the video environment

Run in the same terminal as the video example. The default recommended backend is NVIDIA hardware decoding:

```
python3 "$NORTHSDK_PREFIX/examples/python/check_video_env.py" --backend nvdec
```

The script checks local SDK loading, GStreamer, video plugins and GPU/driver dependencies; C++ users can run it too. All checks reporting `OK` and a final `PASS` mean dependencies are available. Receive video next to verify decoding. On `FAIL`, check the current package environment, library/plugin completeness and driver status.

To check software decoding, pass `--backend software` to the script. Select the capture or preview program's decoder separately, as shown next and in Example Code Reference.

Receive the first video stream

After the dependency check passes and the robot head-left camera is online, run a single read-only capture for five seconds without saving images:

```
"$NORTHSDK_PREFIX/bin/north_sdk_video_jpeg" \
  "$NORTHSDK_ENDPOINT" --no-save 0 85 --seconds=5 --nvdec
```

`0` selects head-left and `85` is JPEG quality. At completion, check that JPEG count is greater than zero and no subscription/decoder errors occurred. If no JPEG arrives, check camera publication, network and decoder requirements. For software decoding, replace `--nvdec` with `--software` after passing the corresponding environment check.

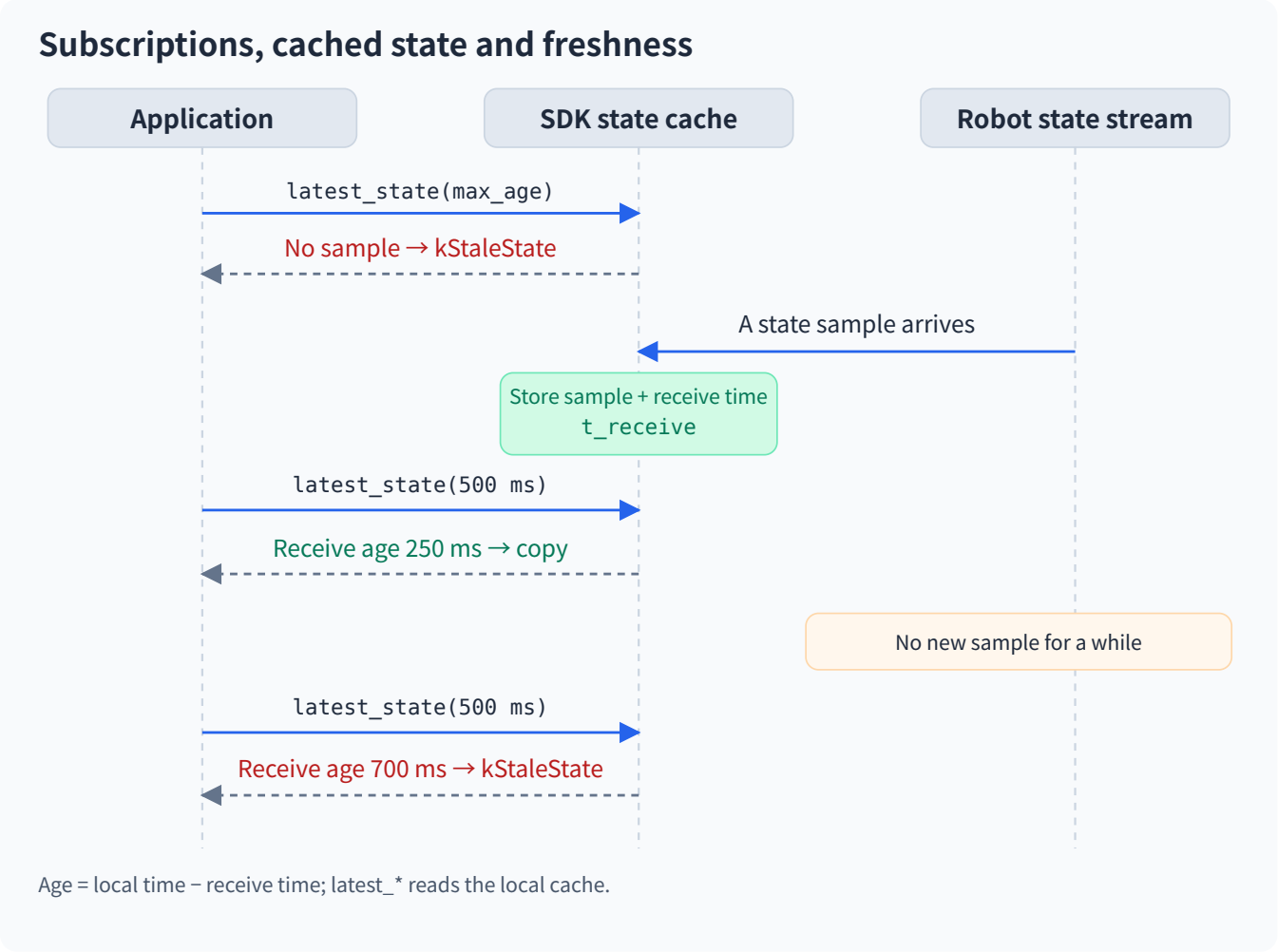
For browser preview, use the scripts in Example Code Reference. See VideoClient for parameters, sources and diagnostics.

Subscriptions and caches

Retain the Subscription returned by on_*(). Destruction or close() cancels it. C++ callback arguments are borrowed references; copy data needed after the callback. Keep callbacks short and move expensive or blocking work to application workers.

latest_*(max_age) returns a local cached copy, aged from local reception. The first call may have no sample or return kStaleState. Wait for a new sample.

Callback policy	Purpose
`kDropOldest`	Default; discard the oldest droppable callback when full
`kLatestOnly`	Coalesce pending samples per subscription for state display
`kReliable`	Apply backpressure instead of local dropping



This example uses max_age=500 ms; 250 ms and 700 ms are local receive ages. latest_* establishes the required state reception without a prior on_* call. A separate on_* subscription also receives callbacks.

Service results

Check the SDK call result first, then the robot service's business result. Monitor heartbeats and error events during operation. C++ uses `Result<T>` / `Status`; Python raises `NorthSDKError`. See Errors and result checking for fields and handling.

Motion control

> **Real motion is not a first-connection test.** SDK validation and the Master gate do not replace a hardware emergency stop, model-specific limits, site risk assessment or the robot-side safety controller.

Before the first motion command, complete at least these checks:

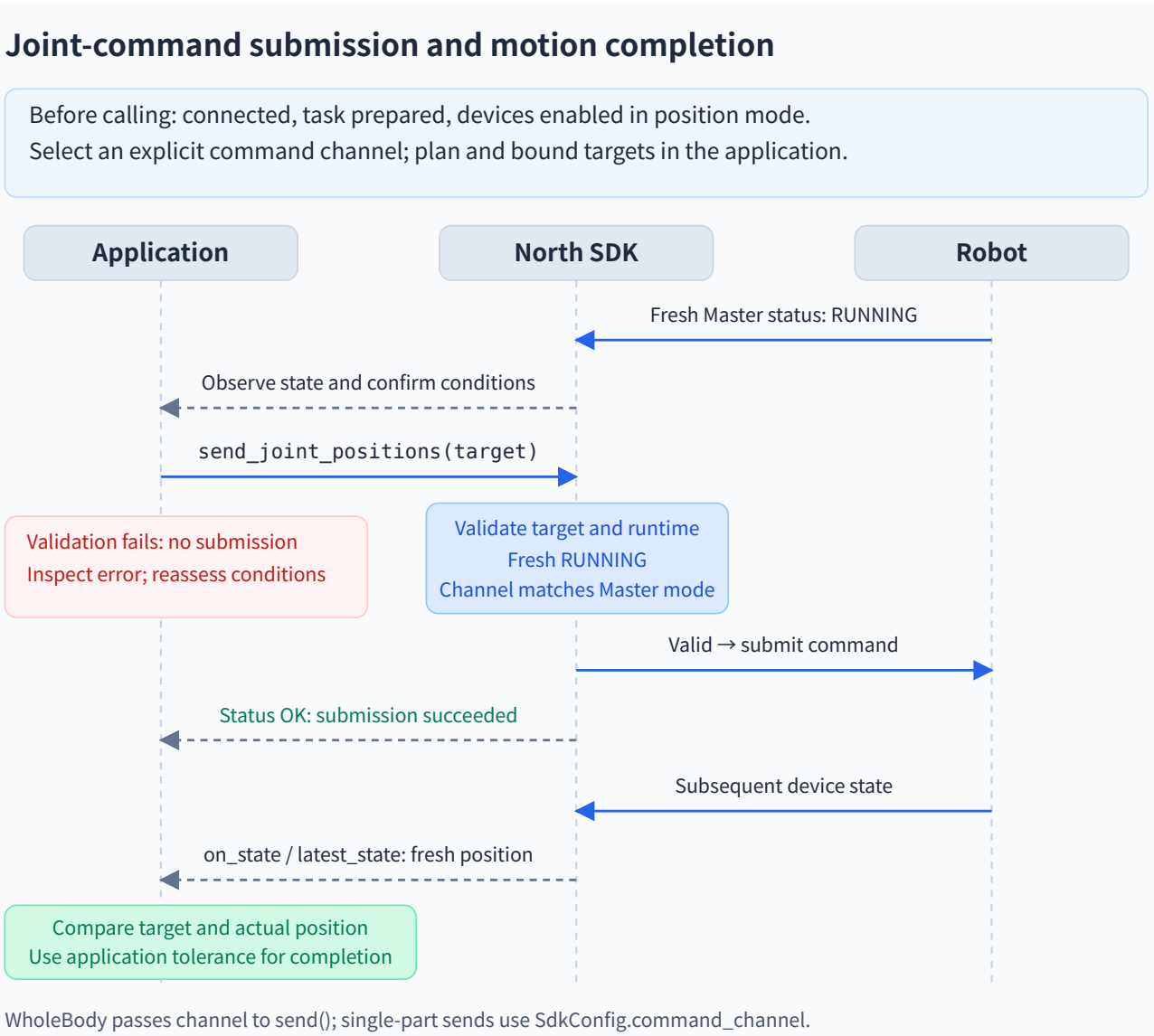
- the emergency stop has been verified and is immediately reachable; the work area is clear and the robot is restrained or supported as required by its model;
- the joint names, array order, positive directions, zero positions, hard/soft limits and recommended velocity/acceleration for this exact model are available;
- fresh feedback is read first and a small target is generated from the current position within model limits;
- the task ID, CommandChannel, Master mode, device enable state and control mode are a supported combination in the delivered robot configuration;
- the control loop stops sending immediately when feedback becomes stale, Master leaves RUNNING, or an SDK/business error occurs;
- reviewed task-stop, emergency-stop and recovery procedures are ready, and the on-site supervisor approves the test.

See Motion protection and trajectory processing for robot-side protection, numeric limits and interpolation settings.

Motion commands require fresh RUNNING Master status. Channel-to-mode mapping is below; follow Prepare and end a task for setup.

Channel	Master mode
`kTeleoperation`	`kTeleoperation`
`kInference`	`kInference`
`kAutomation`	`kAutomation`
`kUniversal`	Any mode; fresh RUNNING is still required

Each arm has seven joints, each hand twenty-two, and the torso eight. Positions are in rad, in fixed device order. Populate complete JointCommand.position only; leave name, velocity and torque empty. A successful send means submission succeeded; determine completion from fresh device feedback.



This sequence illustrates a single-arm position command. Invalid runtime conditions or targets return an SDK error. kUniversal skips mode matching but still requires fresh RUNNING.

Prepare and end a task

The following example sends position targets with the `inference` task.

1. **Configure the channel.** Before SDK creation, set `config.command_channel = north::CommandChannel::kInference` (C++) or `config.command_channel = CommandChannel.INFERENCE` (Python). WholeBody takes its channel on each `send()`.

2. **Check idle state.** Read fresh Master status and require `STANDBY`. Check required device health and confirm other controllers have stopped sending commands. End any existing task using its own procedure first.

3. **Prepare devices.** Call `master().set_mode()` with `OperationMode::kInference` / `OperationMode.INFERENCE`. For each controlled arm select `ControlMode::kJointPosition` / `ControlMode.JOINT_POSITION` and call `enable()`. For torso select

`TorsoControlMode::kPosition` / `TorsoControlMode.POSITION` and enable it. For hands select `HandMode::kPosition` / `HandMode.POSITION`. Check each service result and device feedback.

4. **Start the task.** Call `master().start_task("inference")`, check `MasterResult`, then wait for streamed status with `RUNNING`, `Inference` and `task_id == "inference"`. Resolve device health failures before proceeding.
5. **Send targets.** Generate complete, small, continuous targets from fresh position feedback. Continue checking task ID, Master phase/mode and device feedback. Use the joint ordering in the next section.
6. **End normally.** Stop the control loop. For chassis, keep sending zero velocity until feedback confirms rest. Call `master().stop_task()`, check its business result, wait for status to leave RUNNING and check device feedback before closing the SDK. On feedback or communication failure, use the site stopping procedure.

`start_task(task_id)` takes a case-sensitive task ID. Customer tasks in the standard configuration are listed below; availability follows the delivered configuration. Unconfigured IDs are rejected.

Task ID	Purpose
`inference`	Inference control; used in this example.
`teleoperation`	Teleoperation, with its own preparation flow.
`home`	Homing; actively moves the robot.
`docking`	Docking.

Normal `teleoperation` completion performs homing and leaves RUNNING after homing and recording finalization. **Stopping teleoperation may cause further motion and must not be used as an emergency stop.**

`master().exit_estop()` starts actuator reset and re-enable, including both arms, torso and hand mode recovery. Clear outgoing motion commands and confirm the site is ready before calling it. A successful response means recovery has started. Wait for Master to leave ESTOP, then check health, control modes and fresh position feedback before starting a new control loop.

Joint arrays and coordinates

`joints.name[i]` and `joints.position[i]` refer to the same axis. Commands use the same fixed index, with `name` left empty. Print the complete device state and compare it with the joint definition for the delivered model.

Part	Array indices	State names	Mapping
Each arm	0–6	`joint_0`–`joint_6`	J1–J7; `Side` selects left or right.
Torso	0–4	`torso_0`–`torso_4`	Body J1–J5; J3/J4 are waist pitch/roll joints.
Neck	5–7	`torso_5`–`torso_7`	Neck J6–J8, sent with the body as a complete eight-position command.

Part	Array indices	State names	Mapping
Each hand	0–21	`joint_0–joint_21`	Twenty-two joints.

Positions are in rad. Use the delivered model's joint definitions for each side's zero positions, positive axes and hand joint groups.

Motion protection and trajectory processing

The robot applies command-step checks, limits and interpolation to motion targets. Joint numbers start at 1; SDK indices start at 0.

Command admission and fault protection

Arms and torso process streamed position commands using **the difference between the new target and the previous accepted target**. A frame within every direct-admission threshold is accepted when no catch-up or recovery is pending. Larger steps have their increments limited; steps above Hold reject the entire frame. Axes meeting the fresh-feedback condition below may be exempted from triggering Hold.

Part and checking space	Direct-admission threshold	Hold threshold	Catch-up rate
Each arm, joint angles J1–J7	0.1745 rad per joint	J1–J4: 0.3491 rad; J5–J7: 0.5236 rad	J1–J7: 2.0944, 2.0944, 3.1416, 3.1416, 6.2832, 6.2832, 6.2832 rad/s
Torso, eight motor axes in motor-ID order 1–8	0.0349, 0.0349, 0.0698, 0.0698, 0.0698, 0.0873, 0.0873, 0.0873 rad	0.1047, 0.1047, 0.2094, 0.2094, 0.2094, 0.2618, 0.2618, 0.2618 rad	1.0472, 1.0472, 1.0472, 1.0472, 1.0472, 1.5708, 1.5708, 1.5708 rad/s

During catch-up, all axes use the most restrictive axis's increment scaling factor, with a time allowance capped at 20 ms. Catch-up rates apply during catch-up and recovery.

If feedback is no older than 50 ms and the target-to-feedback difference is within the direct threshold, that axis may avoid Hold caused by a large step from the old target; position commands remain subject to catch-up limits. Recovery from Hold requires checked axes to return within the direct threshold or close to that fresh feedback. **Hold retains the previous target and existing motion may continue.** Torso thresholds are evaluated in motor coordinates.

The robot also checks measured state and hardware faults:

Protection	Default setting and behavior
Arm feedback position	Feedback beyond the position range below expanded outward by 0.0262 rad triggers protection. Application targets must remain within the tabulated position range.
Arm position tracking error	When position safety checking is active, target-to-feedback error above 0.3491 rad for J1–J4 or 0.6982 rad for J5–J7 triggers fault protection.

Protection	Default setting and behavior
Arm motor feedback and hardware faults	Non-finite feedback, drive faults or motor speed above 8.3776 rad/s for motors 1–3 or 26.7035 rad/s for motors 4–7 trigger fault protection.
Chassis command timeout	After 100 ms without a valid motion command, the current velocity command expires. Verify standstill using velocity feedback.

After a fault, inspect heartbeat errors and service responses, then follow the recovery procedure. Self-collision checking is disabled by default; applications must plan collision-free motion.

Position ranges and velocity constraints

Arms

Position targets are clamped per joint as follows. Left and right arms use their respective joint coordinate definitions.

Joint	Lower position (rad)	Upper position (rad)	Joint velocity command clamp (rad/s)
J1	-1.6144	4.2673	8.3776
J2	-2.0944	0.3491	8.3776
J3	-1.2217	4.3633	8.3776
J4	-0.1745	2.2689	11.4605
J5	-1.5708	2.6180	22.9210
J6	-1.0472	1.5708	26.7035
J7	-0.8727	0.8727	26.7035

The velocity column caps joint velocity commands. Applications plan operating speeds for the task. SDK Arm APIs accept position targets; motor limits and mechanism pose further affect actual motion.

Torso and hands

Torso positions are clamped in the fixed SDK joint order:

Joint	J1	J2	J3	J4	J5	J6	J7	J8
Lower (rad)	0	-2.4435	-0.5236	-0.5236	-3.1416	-1.5708	-0.5236	-0.5236
Upper (rad)	1.4835	0	2.0944	0.5236	3.1416	1.5708	0.5236	0.8727

See TorsoClient for joint names and diagram. Combined waist poses must also satisfy mechanism and motor constraints.

Hand limits follow the delivered model's joint table. Default speed and current coefficients are 0.5 and 0.6 respectively; both are dimensionless.

Chassis

Parameter	Default value	Meaning
`vx` / `vy` acceleration	10 m/s² each	Limits body-frame x and y acceleration separately.
`vtheta` angular acceleration	10 rad/s²	Yaw velocity slew limit.
`extension_velocity` acceleration	0.5 m/s²	Stretch-axis velocity slew limit.
Stretch speed	0.05 m/s	Stretch synchronization speed cap.
Stretch length	0.357–0.477 m	Mechanism stretch range; the SDK controls stretch through `extension_velocity`.
Wheel motor speed	31.4159 rad/s	Motor cap; closed-loop wheel control caps speed at 20.9440 rad/s. The stricter limit applies.
Steering angle	±2.7925 rad	Hard limit: ±2.7925 rad. Steering-angle selection uses a ±2.3562 rad soft band.

Steering alignment and stretch state can further reduce wheel speed. Determine `vx/vy/vtheta` ranges from chassis motion constraints. After sending zero velocity, confirm standstill using velocity feedback.

Smoothing continuous position commands

Arms and torso use quintic interpolation to convert streamed position targets into higher-rate control targets. Applications continuously send smooth targets.

Parameter	Each arm	Torso
Control period	1 ms (1000 Hz)	2 ms (500 Hz)
Nominal command period	20 ms	20 ms
Interpolation velocity parameter	10 rad/s	1.0472 rad/s (motor space)
Interpolation acceleration parameter	30 rad/s²	2.0944 rad/s² (motor space)

The nominal period is an interpolation reference and allows variations in send intervals. Larger target changes may extend execution time. Interpolation parameters, catch-up rates and joint velocity limits act at different stages. Applications plan velocity, acceleration and jerk.

The robot connects each segment with a quintic polynomial, matching position, velocity and acceleration at both endpoints:

$$q(t) = a0 + a1\,t + a2\,t^2 + a3\,t^3 + a4\,t^4 + a5\,t^5,\quad 0 \le t \le T$$

Here q is position (rad), T is segment time (s), and the six coefficients are determined by endpoint position, velocity and acceleration. New streamed targets continue from the current motion state.

Smoothing a position target

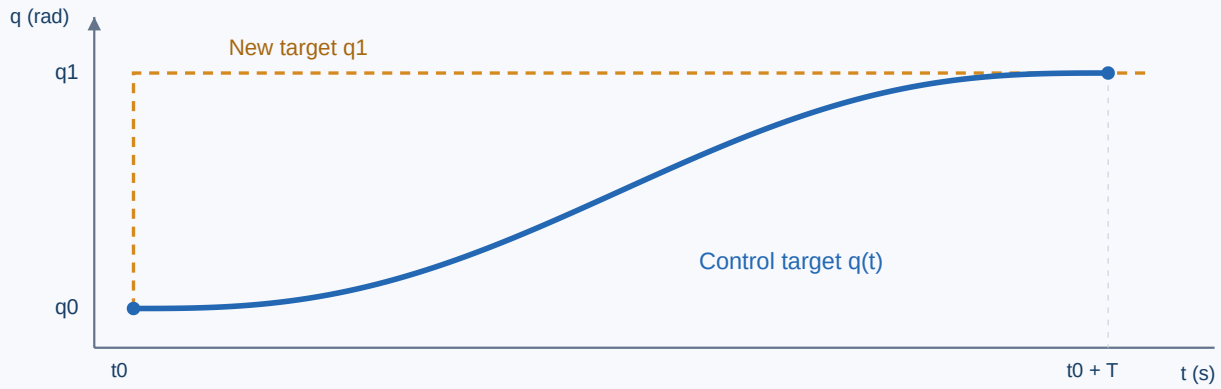


Illustration: zero endpoint velocity and acceleration; streamed motion continues from the current state.

The figure shows a single segment with zero endpoint velocity and acceleration. With `s=t/T`, its position is `q(t)=q0+(q1−q0)(10s³−15s⁴+6s⁵)`. Continuous motion uses the actual endpoint derivatives.

Segment time is estimated from the command period and extended for larger target changes:

```
T0 = clamp(1.5 P, Tc, 0.2 s)
T  = max(T0, 1.875 × max_i |q1[i] − q0[i]| / vmax)
```

P is the estimated command period, `Tc` the control period, and `vmax` a positive interpolation velocity parameter. Position and velocity must use the same coordinate space. The 0.2 s cap applies to timing estimation; actual T may be longer. Velocity and acceleration parameters clamp derivative estimates and outputs. Applications must still check velocity, acceleration and jerk over the complete trajectory.

After a stream stops, the final position is retained and output velocity and acceleration decay within 50 ms. Confirm standstill through device feedback.

Hands receive the latest complete target directly; applications ensure target continuity. Hand feedback without progress for more than 300 ms is marked stale. Existing targets may continue executing after sending stops; confirm standstill through device feedback.

Shutdown and reconnect

Finish the application task-stop workflow, then close subscriptions and the SDK. `close()` cancels subscriptions, clears caches and releases this object's time-sync session. After the last time-sync session closes, the previous system time services are restored. Recreate user subscriptions after reconnecting. Python can use `with NorthSDK(config)` to connect and close automatically.

Troubleshooting

Use this section for data reception, service calls and motion admission after application startup. See First-run troubleshooting for startup/library/import failures and Video environment setup for video dependencies and decoding.

Symptom	Check
No state output or silent stream	Subscription lifetime, installed/online device, robot-side publication and endpoint
Cache read failure	Sample reception, max_age and continued device updates
Call error or service rejection	Check the SDK error first, then response success/result and message; see Errors and result checking
Device fault during operation	Stop sending motion targets. Record the subsystem, error_code, fault_codes and Master reason. Resolve emergency-stop or connectivity causes before device recovery; contact support with these details and software versions for persistent faults.
Time-sync failure or loss	Check the wired robot connection, host setup and diagnostics; see Time synchronization.
Motion rejected	Supported fields, complete targets, command_channel, Master phase and mode

Software Service Interface

API conventions

This chapter covers SDK subscriptions, caches, Actions and synchronous Services. Classes are in namespace `north`. Declarations and public data types follow; include `<north_sdk/NorthSDK.h>` when compiling an application.

Common parameters and results

Object	Usage rule
`Status`	SDK result for connect and Actions; check ok()/bool
`Result<T>`	Check bool before value; on failure inspect error.code/message
`BasicResponse`	After Result succeeds, check success and message
`MasterResponse`	Check result against kOk; before/after are service snapshots
`Header`	`stamp_ns` is source time and `system_time_ns` is remote system time; their domains may differ. `seq` is a sequence number, `key` identifies the source, and `frame_id` identifies a source or coordinate frame.
`Side`	kLeft/kRight; Python LEFT/RIGHT; invalid values are rejected

Default units are SI: m, rad, m/s, rad/s, N, Nm, A and V, with temperature in degrees Celsius. Time fields use explicit _ns/_ms/_s units; power clocks use clock_domain. Arrays use fixed device joint order and quaternion fields are w/x/y/z.

These rules apply to state/event subscriptions using `SubscribeOptions` and to `latest_*` cache reads. See VideoClient and AlignmentClient for their specialized options.

Common parameter	Type, default and rules
`callback`	Required, nonempty C++ `std::function<void(const T&)>` or Python callable. The C++ reference is callback-scoped; copy data needed afterward.
`options`	`SubscribeOptions`, default DropOldest; controls local callback queuing. Python defaults to `SubscribeOptions()`.
`max_age` / `max_age_ms`	Required C++ milliseconds; Python int defaults to 1000 ms. Must be nonnegative. Measures elapsed time since SDK reception on this computer, independently of message timestamps.

Retain the subscription returned by `on_*`. Destruction, `close()`, or leaving a Python subscription context cancels it. SDK worker callbacks should remain short; no new callback starts after `close()` returns.

`latest_*` automatically receives state and returns a cached copy without a prior subscription. Missing or expired samples return `kStaleState`, negative age returns `kInvalidArgument`. Choose the age from the device publication period and application latency requirements. C++ checks `Result<T>`; Python returns data or raises `NorthSDKError`.

SdkConfig defaults

Field	Default	Meaning
`zenoh_config_path`	Empty	Native Zenoh configuration file
`connect_endpoints`	{}	Supply endpoints explicitly or through the file
`connect_timeout`	5000 ms	Wait for one communication-session connection attempt
`service_timeout`	6000 ms	Ordinary service wait configuration
`master_timeout`	15000 ms	Master services
`torso_timeout`	20000 ms	Torso services
`callback_queue_capacity`	256	Bounded callback queue capacity in tasks
`master_status_max_age`	500 ms	Maximum receive age for motion admission; must be positive
`command_channel`	kUnspecified	Set explicitly for single-part sends
`time_sync`	Enabled, software mode	Prepares gPTP after connection; see Time-sync parameters.

Errors and result checking

Check SDK call errors, robot service outcomes and runtime device faults separately:

Layer	How to obtain it	Handling
SDK call result	C++: `Result<T>.error().code/message` or `Status.code()/message()`; Python: catch `NorthSDKError`, read `.code` and the exception text	Resolve argument, connection, timeout or SDK motion-gate issues first. Codes are listed below.
Robot service result	Ordinary response `success/message`; Master `result/message`; configuration service `result` and `diagnostics`	A received response still needs a business-result check. Master success is `MasterResult::kOk`; configuration success is `ConfigResult::kOk`.
Robot runtime fault	Part Client `on_heartbeat()`: `error_code/fault_mask/fault_codes`; Master `on_error()` / `on_degradation()`: `error_code/reason/device_errors`	Stop sending motion targets and record the subsystem, fault codes and reason. Resolve site causes before reset; contact support for persistent faults.

Interpret device faults using the corresponding device definitions: `HandServiceResponse.sdk_error_code` comes from the hand device SDK, and `PowerCommandResponse.error_code` comes from the power service. North SDK call errors use `ErrorCode`; `kRemoteFailure` indicates a communication error reply.

ErrorCode	Handling
`kOk`	Success
`kInvalidArgument`	Correct lengths, finite values, enum values, confirmations or configuration
`kNotConnected`	Connect successfully before calling
`kUnavailable`	Check device service availability and connectivity
`kTimeout`	Response wait timed out; inspect results before retrying side effects
`kSerialization`	Check matching software versions and retain error details and logs
`kTransport`	Retain the Zenoh/network error and inspect connectivity
`kStaleState`	Missing, malformed or stale state; wait for fresh state
`kOutcomeUnknown`	Master outcome unknown; inspect state without automatic retry
`kRemoteFailure`	Zenoh error reply received; inspect message
`kInternal`	Retain error details and logs, and contact support
`kMotionNotAllowed`	Check Master phase and channel/mode
`kNotSupported`	The current version or device does not support the requested operation or parameter; check arguments and version applicability
`kTimeSyncUnavailable`	Check host-component installation and the service error message.
`kTimeSyncPermission`	Use the authorized account; software mode also needs system-time authorization.
`kTimeSyncUnsupported`	Check the selected mode and NIC timestamping capabilities.
`kTimeSyncConflict`	Check other time services or conflicting synchronization sessions.
`kTimeSyncTimeout`	Check the link and interface, and whether synchronization continues converging.
`kTimeSyncLost`	Synchronization is not ready or has been lost; wait for readiness.

After a timeout or error reply, the server may already have performed the operation. Inspect actual state before retrying.

Python mapping

C++	Python
`NorthSDK::Create(config)` + `connect()`	`NorthSDK(config)` + `connect()` / `with NorthSDK(config)`
`Result<T>`	Return T directly; SDK failures raise NorthSDKError
`Status`	Return None on success; raise NorthSDKError on failure

C++	Python
`Side::kLeft` / `CommandChannel::kTeleoperation`	`Side.LEFT` / `CommandChannel.TELEOPERATION`
`connect_timeout` / `master_status_max_age`	`connect_timeout_ms` / `master_status_max_age_ms`
`latest_state(std::chrono::milliseconds(200))`	`latest_state(200)`
`sdk->connected()` / `subscription.active()`	`sdk.connected` / `subscription.active`
`std::optional<T>`	Value or None
`std::array / std::vector`	Python sequences
Binary std::string fields	`bytes`

An incompatible Python argument type or length may also raise `TypeError`.

Common Python signatures

Apart from the differences below, method names match C++ and enums use uppercase underscores. Subscription `options` may be omitted, defaulting to `SubscribeOptions()`. Synchronous services return response objects directly.

Python call	Return
`NorthSDK(config)`; `sdk.connect()`; `sdk.close()`	SDK object; the latter two return `None` on success.
`sdk.time_sync_diagnostics()`; `sdk.robot_time_ns()`	`TimeSyncDiagnostics`; integer nanoseconds.
`client.latest_state(max_age_ms=1000)`	Part state; `latest_status/latest_imu/latest_battery` also use `max_age_ms=1000`.
`client.on_state(callback, options=SubscribeOptions())`	`Subscription`; Master uses `on_status`.
`sdk.master().on_heartbeat(subsystem, callback, options=SubscribeOptions())`	`Subscription`.
`sdk.master().set_mode(mode)`; `start_task(task_id)`; `stop_task(status=0)`	`MasterResponse`; check `result`.
`sdk.arm(side).send_joint_positions(positions)`	`None`; seven positions. The hand method of the same name takes twenty-two.
`sdk.torso().send_joint_command(command)`	`None`; `command` is a `JointCommand`.
`sdk.whole_body().send(channel, command)`	`None`; `command` is a `WholeBodyCommand`.
`sdk.video().on_jpeg(source, callback, options=VideoOptions())`	`Subscription`; `diagnostics(source)` returns `VideoDiagnostics`.
`sdk.alignment().on_frame(options, callback, callback_options=SubscribeOptions())`	`Subscription`; `diagnostics(subscription)` returns `AlignmentDiagnostics`.
`sdk.config().get(plugin_id="")`; `set(plugin_id, operations)`	`ConfigGetResponse` / `ConfigSetResponse`.

Python tolerances are integer ns (`max_skew_ns`); waits and cache ages are integer ms (such as `timeout_ms` and `wait_timeout_ms`). `sdk.connected`, `subscription.active` and `sample.matched` are properties, without parentheses.

Public capability scope

The following table summarizes the capabilities of each Client.

Client	Public capability
Config	Instance discovery, descriptors, persistent SET/RESET
Master	Status, hardware health, events, heartbeat, task and lifecycle services
WholeBody	Multi-part position/velocity commands and command mirror
Chassis	State, heartbeat, cache, velocity, Motion/Force mode, enable and reset
Torso	State, heartbeat, cache, complete positions, enable, reset, mode and action cancel
Arm	Side-specific state, heartbeat, cache, complete positions, enable, mode, estop, reset and action cancel
Hand	State, tactile/e-skin, complete positions, modes, tactile/e-skin calibration, zero-calibration query/stop
Sensors	IMU, ultrasonic, tactile skin, Lidar and tactile-skin zeroing
Power	Board/battery state, heartbeat, read-only queries and explicitly confirmed power management
Video	Four AU decoders, JPEG callbacks and diagnostics; requires a video-enabled package and its runtime dependencies
Alignment	Source-timestamp matching and diagnostics for video, device and Master state

Capabilities depend on installed and enabled robot devices. Check device state, SDK errors and business responses during operation.

Client APIs

Each on_* returns `Result<Subscription>`; latest takes a maximum age. Usage rules and API declarations follow. For application workflows see Application Development.

NorthSDK

Create and connect an instance, then use accessors to obtain Clients. version() reports the version; connected() reports local connection state.

```cpp
class NorthSDK {
public:
    static Result<std::unique_ptr<NorthSDK>> Create(const SdkConfig& config);
    ~NorthSDK();
    NorthSDK(const NorthSDK&) = delete;
    NorthSDK& operator=(const NorthSDK&) = delete;

    Status connect();
    void close();
    bool connected() const noexcept;
    CallbackDiagnostics callback_diagnostics() const;
    TimeSyncDiagnostics time_sync_diagnostics() const;
    Result<std::int64_t> robot_time_ns() const;
    static const char* version() noexcept;

    ConfigClient config();
    MasterClient master();
    WholeBodyClient whole_body();
    ChassisClient chassis();
    TorsoClient torso();
    ArmClient arm(Side side);
    HandClient hand(Side side);
    SensorClient sensors();
    PowerClient power();
    VideoClient video();
    AlignmentClient alignment();

};
```

API	Parameters	Result and purpose	Notes
`Create(config)`	`config: SdkConfig`	C++ returns `Result<std::unique_ptr<NorthSDK>>`; creates without connecting	Validates channel, timeouts, and callback capacity. Python uses `NorthSDK(config)` and raises `NorthSDKError` on failure.
`connect()`	None	C++ `Status`; Python returns `None` on success	Establishes communication and prepares configured synchronization. By default a time-sync error retains communication; check `connected` and synchronization diagnostics separately.
`close()`	None	No result	Cancels subscriptions, clears caches, and closes connections. Recreate user subscriptions after reconnecting. Destruction and Python `with` exit also close.
`connected()`	None	`bool`	Reports whether the local communication session is established. Check heartbeats and state streams for device availability.
`callback_diagnostics()`	None	`CallbackDiagnostics`	Returns cumulative dropped/coalesced callback counts and the latest decode error.
`time_sync_diagnostics()`	None	`TimeSyncDiagnostics`	Reads synchronization state, offset and errors; see Time-sync parameters.

API	Parameters	Result and purpose	Notes
`robot_time_ns()`	None	C++ `Result<int64_t>`; Python `int`, in ns	Reads a local clock synchronized to North gPTP. Returns an error if disconnected, disabled or not ready.
`version()`	None	Version string	Static; no SDK instance or connection is required.
`config()`, `master()`, `whole_body()`, `chassis()`, `torso()`, `sensors()`, `power()`, `video()`, `alignment()`	None	Corresponding Client	Clients cannot communicate after the SDK closes.
`arm(side)`, `hand(side)`	`side: Side`	Side-bound Client	Left/Right are from the robot's perspective. An invalid value is rejected by later calls.

`SdkConfig.connect_endpoints` and `zenoh_config_path` select the target. Single-part motion also requires `command_channel` before creation. See SdkConfig defaults.

Subscription

A move-only subscription handle. Retain it to receive callbacks; destruction or `close()` cancels it.

```
class Subscription {
public:
    Subscription();
    ~Subscription();
    Subscription(Subscription&&) noexcept;
    Subscription& operator=(Subscription&&) noexcept;
    Subscription(const Subscription&) = delete;
    Subscription& operator=(const Subscription&) = delete;

    void close();
    bool active() const noexcept;

};
```

API	Parameters	Result and purpose	Notes
`close()`	None	Cancels the subscription	Call explicitly or rely on C++ destruction; Python may use `with subscription:`. No new callback starts after it returns.
`active()`	None	`bool`	Reports whether the handle remains open.

ConfigClient

```
class ConfigClient {
public:
    Result<ConfigGetResponse> get(const std::string& plugin_id = {});
    Result<ConfigSetResponse> set(
        const std::string& plugin_id,
```

```
        const std::vector<ConfigFieldOperation>& operations);
};
```

`get()` queries robot-exposed configuration instances and fields. `set()` persists configuration changes. Use instance and field IDs returned by `get()`.

API	Parameters and return
`get(plugin_id="")`	Empty queries all instances; a non-empty ID queries one instance. Returns `ConfigGetResponse`.
`set(plugin_id, operations)`	Takes an instance and a list of field operations. SET carries a value; RESET does not. Returns `ConfigSetResponse`.

Require `result == ConfigResult::kOk`; inspect `diagnostics` on failure. Before writing, check the field descriptor's type, range and application conditions. After writing, inspect `effective_value` and `pending_init`. After a write timeout, call `get()` to confirm the actual result.

See Configuration data for types and fields.

MasterClient

Provides runtime status, health, task events and system services. Use task IDs configured on the robot. on_heartbeat accepts a subsystem name or *. Check Master response.result, then observe state after transitions.

```
class MasterClient {
public:
    Result<Subscription> on_status(std::function<void(const MasterStatus&)> callback,
                                   const SubscribeOptions& options = {});
    Result<Subscription> on_hardware_health(
        std::function<void(const SystemHardwareHealth&)> callback,
        const SubscribeOptions& options = {});
    Result<Subscription> on_task_start(std::function<void(const MasterEvent&)> callback,
                                       const SubscribeOptions& options = {});
    Result<Subscription> on_task_stop(std::function<void(const MasterEvent&)> callback,
                                      const SubscribeOptions& options = {});
    Result<Subscription> on_degradation(std::function<void(const MasterEvent&)> callback,
                                        const SubscribeOptions& options = {});
    Result<Subscription> on_error(std::function<void(const MasterEvent&)> callback,
                                  const SubscribeOptions& options = {});
    Result<Subscription> on_heartbeat(const std::string& subsystem,
                                      std::function<void(const Heartbeat&)> callback,
                                      const SubscribeOptions& options = {});
    Result<MasterStatus> latest_status(std::chrono::milliseconds max_age);

    Result<MasterResponse> set_mode(OperationMode mode);
    Result<MasterResponse> start_task(const std::string& task_id);
    Result<MasterResponse> stop_task(std::int32_t status = 0);
    Result<MasterResponse> reset();
    Result<MasterResponse> enter_ota();
    Result<MasterResponse> exit_ota();
    Result<MasterResponse> enter_estop();
    Result<MasterResponse> exit_estop();
    Result<ConfigSetResponse> set_health_requirement(const std::string& subsystem,
                                                     HealthRequirement requirement);
    Result<ConfigSetResponse> set_safety_guard_enabled(const std::string& guard_id, bool enabled);
    Result<BasicResponse> prepare_teleoperation();

};
```

API	Parameters	Result data	Purpose and constraints
`on_status(callback, options)`	Common subscription parameters	`MasterStatus`	Subscribe to Master phase, operating mode, task ID, and degradation state.
`on_hardware_health(callback, options)`	Common subscription parameters	`SystemHardwareHealth`	Subscribe to subsystem health, safety guards, and host resource summary.
`on_task_start(callback, options)`	Common subscription parameters	`MasterEvent`	Subscribe to task-start acceptance, active state, and reason.
`on_task_stop(callback, options)`	Common subscription parameters	`MasterEvent`	Subscribe to task-stop events and final status.
`on_degradation(callback, options)`	Common subscription parameters	`MasterEvent`	Subscribe to system degradation events.
`on_error(callback, options)`	Common subscription parameters	`MasterEvent`	Subscribe to Master errors and per-device error details.
`on_heartbeat(subsystem, callback, options)`	`subsystem: string` and common subscription parameters	`Heartbeat`	Subscribe to a subsystem such as `"lidar"`; use `"*"` for all. A specific subsystem name must be non-empty and contain no wildcard/path special characters.
`latest_status(max_age)`	Maximum local receive age	`MasterStatus`	Actively read fresh Master cache; absent or expired data returns `kStaleState`.
`set_mode(mode)`	`OperationMode`: Teleoperation, Inference or Automation	`MasterResponse`	Changes Master operation mode; robot state determines whether switching is allowed. See Application Development for the task preparation sequence.
`start_task(task_id)`	Non-empty, case-sensitive task ID, such as `inference`, `teleoperation`, `home` or `docking`; availability follows the delivery configuration	`MasterResponse`	Requests task start. After an accepted response, still wait for streamed RUNNING state.
`stop_task(status=0)`	32-bit task exit status, default 0; other values are task-defined	`MasterResponse`	Requests normal task completion. `teleoperation` quiesces remote control, performs homing and finalizes recording while remaining RUNNING. An accepted response does not mean standstill. Observe status and feedback for other tasks too. Do not use as an emergency stop.
`reset()`	None	`MasterResponse`	Requests the Master recovery flow; allowed phases and result are decided by the robot state machine.

API	Parameters	Result data	Purpose and constraints
`enter_ota()` / `exit_ota()`	None	`MasterResponse`	Enter or leave OTA phase, affecting tasks and device operations.
`enter_estop()`	None	`MasterResponse`	Requests system emergency stop; check device feedback to confirm the outcome.
`exit_estop()`	None	`MasterResponse`	Starts asynchronous recovery: resets and re-enables both arms and torso, and returns both hands to position mode. Success means recovery started. Wait for Master to leave ESTOP and check device health and feedback. Confirm the site is ready first; do not restart the old control loop directly.
`set_health_requirement(subsystem, requirement)`	Subsystem ID and `HealthRequirement` such as `OK`, `WARNING`, or `OFF`	`ConfigSetResponse`	Persists Master health admission through Config and returns `ConfigSetResponse`.
`set_safety_guard_enabled(guard_id, enabled)`	Guard ID and enabled flag	`ConfigSetResponse`	Persists a safety-guard setting through Config and returns `ConfigSetResponse`; restrict to authorized management.
`prepare_teleoperation()`	None	`BasicResponse`	Prepares the teleoperation path in Master Teleoperation mode. Start the `teleoperation` task once preparation is complete. Check `BasicResponse.success/message`.

Master services require a connected SDK and use `master_timeout` (15000 ms by default). C++ checks `Result<MasterResponse>` followed by `result == MasterResult::kOk`; Python checks `MasterResult.OK` on the returned response and raises `NorthSDKError` on SDK failures. Check `BasicResponse.success/message` for `prepare_teleoperation()`. Health-requirement and safety-guard settings persist through Config; check `ConfigResult::kOk` and diagnostics.

For `set_mode`, Teleoperation, Inference and Automation correspond to the same-named CommandChannels. Universal skips mode matching only when sending and still requires fresh RUNNING state. `before/after` are response snapshots and `message` explains the business result; continue observing `on_status` or `latest_status` to confirm actual mode and phase. See Master mode service.

Common SDK errors include `kNotConnected`, `kInvalidArgument`, `kTimeout`, `kTransport`, `kRemoteFailure` and `kOutcomeUnknown`. See Common parameters and results for subscription and cache rules.

WholeBodyClient

Combines position targets and chassis velocity commands for multiple parts.

```
class WholeBodyClient {
public:
    Status send(CommandChannel channel, const WholeBodyCommand& command);
    Result<Subscription> on_command(std::function<void(const WholeBodyCommand&)> callback,
                                    const SubscribeOptions& options = {});

};
```

API	Parameters	Result and purpose	Constraints
`send(channel, command)`	`channel: CommandChannel`; `command: WholeBodyCommand`	C++ `Status`; Python returns `None` on success	Channel must be Teleoperation, Inference, Automation, or Universal. Supply at least one part; only populated optionals update. Every part is locally validated. Parts execute separately; check feedback for each part.
`on_command(callback, options)`	Common subscription parameters	`WholeBodyCommand` callback	Records the combined command mirror; read device state for execution results.

The `channel` argument of each `send()` call selects the sending channel. It still requires fresh RUNNING Master state.

ChassisClient

Velocity uses m/s and rad/s and must be finite. `set_mode` explicitly selects overall Motion/Force mode; `mode` reads current mode from cached chassis heartbeat. Switching requires an idle chassis outside OTA.

```
class ChassisClient {
public:
    Result<Subscription> on_state(std::function<void(const ChassisState&)> callback,
                                  const SubscribeOptions& options = {});
    Result<Subscription> on_heartbeat(std::function<void(const Heartbeat&)> callback,
                                      const SubscribeOptions& options = {});
    Result<ChassisState> latest_state(std::chrono::milliseconds max_age);
    Status send_velocity(const ChassisVelocity& command);
    Result<ChassisModeResponse> set_mode(ChassisControlMode mode);
    Result<ChassisControlMode> mode(
        std::chrono::milliseconds max_age = std::chrono::milliseconds(500));
    Result<BasicResponse> enable(bool enabled = true);
    Result<BasicResponse> reset();

};
```

API	Parameters	Result data	Purpose and constraints
`on_state(callback, options)`	Common subscription parameters	`ChassisState`	Subscribe to chassis velocity, joint, electrical, and mode state.
`on_heartbeat(callback, options)`	Common subscription parameters	`Heartbeat`	Subscribe to lifecycle, online masks, faults, and rates.

API	Parameters	Result data	Purpose and constraints
`latest_state(max_age)`	Maximum local receive age; Python default 1000 ms	`ChassisState`	Missing or expired data returns `kStaleState`.
`send_velocity(command)`	`ChassisVelocity`	`Status` / Python `None`	`vx`, `vy`, and `extension_velocity` are m/s; `vtheta` is rad/s. All values must be finite. Uses the single-part command channel and Master gate.
`set_mode(mode)`	`ChassisControlMode::kMotion` / `kForce`	`ChassisModeResponse`	Switches overall motion/force mode and returns `success/message`, previous_mode, and current_mode. After switching, confirm the device is enabled.
`mode(max_age=500ms)`	Maximum local heartbeat age	`ChassisControlMode`	Reads fresh cached heartbeat without a service request. First call or expired heartbeat returns `kStaleState`; Python uses `max_age_ms`.
`enable(enabled=true)`	`enabled: bool`	`BasicResponse`	Requests enable or disable; inspect business response and subsequent state.
`reset()`	None	`BasicResponse`	Requests chassis reset; determine enable/mode changes from device feedback.

Mode switching requires an idle chassis outside OTA freeze. Heartbeat period may exceed the default 500 ms window, so applications may choose a larger age while still handling stale results. Velocity commands do not apply authoritative model limits in the SDK; use delivered limits. For normal stopping, keep sending zero velocity and use fresh `ChassisState.velocity` to require translation and rotation within application-defined rest tolerances for the required duration.

TorsoClient

Eight joints are ordered body J1–J5 then neck J6–J8. Send complete positions. Reset leaves joints disabled; enable through the application workflow before motion. Services use torso_timeout.

Torso position array order



JointCommand.position must contain all eight entries.

Leave name, velocity and torque empty. J1 maps to position[0].

Array indices start at zero: for example, J6 is position[5]. The figure shows array order; joint positions are in rad.

```
class TorsoClient {
public:
    Result<Subscription> on_state(std::function<void(const TorsoState&)> callback,
                                  const SubscribeOptions& options = {});
    Result<Subscription> on_heartbeat(std::function<void(const Heartbeat&)> callback,
                                      const SubscribeOptions& options = {});
    Result<TorsoState> latest_state(std::chrono::milliseconds max_age);
    Status send_joint_command(const JointCommand& command);
    Result<BasicResponse> enable(bool enabled = true);
    Result<BasicResponse> reset();
    Result<ModeResponse> set_mode(TorsoControlMode mode);
    Result<ModeResponse> set_mode(std::uint32_t mode);
    Result<BasicResponse> cancel_waypoints();

};
```

API	Parameters	Result data	Purpose and constraints
`on_state(callback, options)`	Common subscription parameters	`TorsoState`	Subscribe to eight-joint state and electrical data.
`on_heartbeat(callback, options)`	Common subscription parameters	`Heartbeat`	Subscribe to torso health and rates.
`latest_state(max_age)`	Maximum local receive age; Python default 1000 ms	`TorsoState`	Missing or expired data returns `kStaleState`.
`send_joint_command(command)`	`JointCommand`	`Status` / Python `None`	Supports exactly eight position values in rad; name, velocity, and torque must be empty. Uses the single-part channel and Master gate.
`enable(enabled=true)`	`bool`	`BasicResponse`	Requests enable/disable and uses `torso_timeout`.
`reset()`	None	`BasicResponse`	Requests reset. The current contract leaves joints disabled after reset; explicitly re-enable before motion.
`set_mode(mode)`	Prefer `TorsoControlMode`; a `uint32_t` overload remains	`ModeResponse`	Switches Position/MIT device mode; check `success`, `previous_mode`, and `current_mode`.

API	Parameters	Result data	Purpose and constraints
`cancel_waypoints()`	None	`BasicResponse`	Requests waypoint cancellation; continue reading state to confirm the outcome.

All torso services use `SdkConfig.torso_timeout`, 20000 ms by default.

ArmClient

arm(Side) selects an arm with seven joints. Mode, enable, estop and reset are explicit services. Check business results and device state after mode changes.

```
class ArmClient {
public:
    Side side() const noexcept;
    Result<Subscription> on_state(std::function<void(const ArmState&)> callback,
                                  const SubscribeOptions& options = {});
    Result<Subscription> on_heartbeat(std::function<void(const Heartbeat&)> callback,
                                      const SubscribeOptions& options = {});
    Result<ArmState> latest_state(std::chrono::milliseconds max_age);
    Status send_joint_positions(const std::array<float, 7>& positions);
    Status send_joint_command(const JointCommand& command);
    Result<BasicResponse> enable(bool enabled = true);
    Result<ModeResponse> set_mode(ControlMode mode);
    Result<BasicResponse> estop(const std::string& reason = {});
    Result<BasicResponse> reset();
    Result<BasicResponse> cancel_waypoints();

};
```

Select the arm through `sdk.arm(Side)`; `side()` returns the Client's bound side.

API	Parameters	Result data	Purpose and constraints
`side()`	None	`Side`	Returns the side bound when the Client was created.
`on_state(callback, options)`	Common subscription parameters; default DropOldest	`ArmState` callback	Receive this arm's state for feedback, monitoring or recording; retain the subscription.
`on_heartbeat(callback, options)`	Common subscription parameters	`Heartbeat`	Subscribe to lifecycle, joint online masks, faults, and rates for this arm.
`latest_state(max_age)`	Required C++ milliseconds; Python `max_age_ms=1000`, nonnegative	`ArmState`	Returns a copy subject to local receive age and automatically establishes reception; missing/expired samples return `kStaleState`.
`send_joint_positions(positions)`	C++ `std::array<float, 7>`; Python numeric sequence of length 7, finite values in rad	`Status` / Python `None`	Submits a complete target for this arm in model-defined joint order; no name reordering, sparse updates, velocity or torque control.

API	Parameters	Result data	Purpose and constraints
`send_joint_command(command)`	`JointCommand`	`Status` / Python `None`	Same constraints as `send_joint_positions`: exactly seven positions; name, velocity, and torque must be empty.
`enable(enabled=true)`	`bool`	`BasicResponse`	Requests enable/disable; check `success/message` and observe state.
`set_mode(mode)`	`ControlMode`	`ModeResponse`	Requests a device control mode. Position commands require the compatible mode defined by the robot procedure. Master `OperationMode` is a separate concept.
`estop(reason="")`	Optional reason string	`BasicResponse`	Requests an emergency stop for this arm. `reason` records the emergency-stop cause.
`reset()`	None	`BasicResponse`	Requests reset; reconfirm mode, enable state, heartbeat, and state afterward.
`cancel_waypoints()`	None	`BasicResponse`	Requests cancellation of an arm waypoint task; observe feedback to confirm.

See Common parameters and results for subscription and cache rules.

Position sending requires `SdkConfig.command_channel` set at creation, a connected SDK, Master RUNNING no older than `master_status_max_age`, matching Teleoperation/Inference/Automation channel and Master mode, and an enabled device in a compatible position mode. The SDK validates length, finite values and motion admission. See Motion control for target preparation.

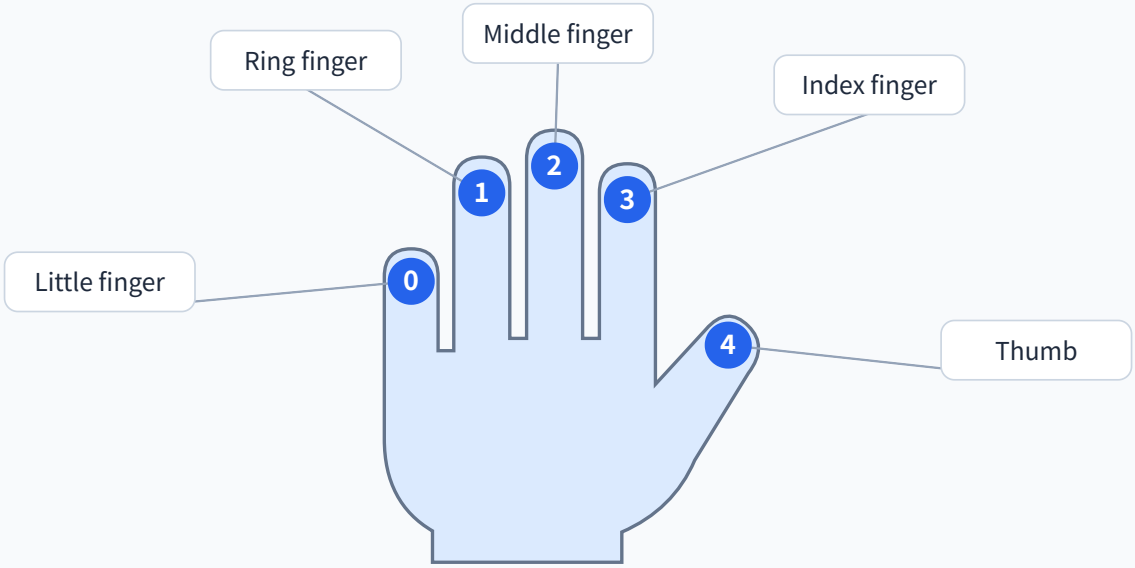
Common send errors are `kInvalidArgument`, `kNotConnected`, `kStaleState`, `kMotionNotAllowed` and `kTransport`. Compare fresh `ArmState.joints.position` against the target using application tolerance and timeout. Stop sending on stale feedback, Master leaving RUNNING or errors. See Send a single-arm target.

Ordinary Arm services use `service_timeout`, 6000 ms by default. Use subsequent state to confirm mode, enable, reset and estop outcomes.

HandClient

Each hand has twenty-two joints. Tactile finger channels 0–4 run from little finger to thumb, with kRaw/kDeform images. Python image payloads are bytes. Tactile calibration accepts enabled=true only. Zero-calibration status query and stop are available. Check success/message.

Hand tactile data: finger channel numbers



Both hands use the same numbers for the same finger.

`sdk.hand(Side::kLeft).on_tactile_image(4, kind, callback)`

Select the hand with hand(Side), then select the finger. Both hands use 0=little, 1=ring, 2=middle, 3=index, 4=thumb. finger=4 selects the thumb of the selected hand.

```
class HandClient {
public:
    Side side() const noexcept;
    Result<Subscription> on_state(std::function<void(const HandState&)> callback,
                                  const SubscribeOptions& options = {});
    Result<Subscription> on_heartbeat(std::function<void(const Heartbeat&)> callback,
                                      const SubscribeOptions& options = {});
    Result<Subscription> on_tactile_force(std::uint32_t finger,
                                          std::function<void(const TactileForce&)> callback,
                                          const SubscribeOptions& options = {});
    Result<Subscription> on_tactile_image(std::uint32_t finger, TactileImageKind kind,
                                          std::function<void(const CompressedImage&)> callback,
                                          const SubscribeOptions& options = {});
    Result<Subscription> on_eskin_force(std::function<void(const HandEskinForce&)> callback,
                                        const SubscribeOptions& options = {});
    Result<Subscription> on_eskin_raw(std::function<void(const HandEskinRaw&)> callback,
                                      const SubscribeOptions& options = {});
    Result<HandState> latest_state(std::chrono::milliseconds max_age);
    Status send_joint_positions(const std::array<float, 22>& positions);
    Status send_joint_command(const JointCommand& command);
    Result<HandServiceResponse> set_mode(HandMode mode);
    Result<HandServiceResponse> set_mode(std::uint32_t mode);
    Result<HandServiceResponse> calibrate_tactile(bool enabled = true);
    Result<HandServiceResponse> calibrate_eskin();
    Result<HandServiceResponse> zero_calibration_status();
    Result<HandServiceResponse> stop_zero_calibration();

};
```

Select the side with `sdk.hand(Side)`. All services use `service_timeout`. For `HandServiceResponse`, inspect `success/message` and, when relevant, result, mode, calibration state, and progress fields.

API	Parameters	Callback/result	Purpose and constraints
`side()`	None	`Side`	Returns the bound hand side.

API	Parameters	Callback/result	Purpose and constraints
`on_state(callback, options)`	Common subscription parameters	`HandState`	Subscribe to 22-joint, electrical, and force information.
`on_heartbeat(callback, options)`	Common subscription parameters	`Heartbeat`	Subscribe to hand health and rates.
`on_tactile_force(finger, callback, options)`	`finger` 0–4	`TactileForce`	Subscribe to six-axis tactile force; 0=little finger and 4=thumb. Out-of-range returns `kInvalidArgument`.
`on_tactile_image(finger, kind, callback, options)`	Finger 0–4 and Raw/Deform kind	`CompressedImage`	Subscribe to raw or deformation tactile images. `format` describes encoding and `data` is binary.
`on_eskin_force(callback, options)`	Common subscription parameters	`HandEskinForce`	Subscribe to fitted e-skin force.
`on_eskin_raw(callback, options)`	Common subscription parameters	`HandEskinRaw`	Subscribe to raw e-skin binary data.
`latest_state(max_age)`	Maximum local receive age; Python default 1000 ms	`HandState`	Missing or expired data returns `kStaleState`.
`send_joint_positions(positions)`	22 finite rad values	`Status` / Python `None`	Submit a complete fixed-order target through the single-part channel and Master gate.
`send_joint_command(command)`	`JointCommand`	`Status` / Python `None`	Supports exactly 22 positions; name, velocity, and torque must be empty.
`set_mode(mode)`	Prefer `HandMode`; a `uint32_t` overload remains	`HandServiceResponse`	Switches hand device mode.
`calibrate_tactile(enabled=true)`	`enabled: bool`	`HandServiceResponse`	The current service accepts only `true`; `false` returns `kInvalidArgument`.
`calibrate_eskin()`	None	`HandServiceResponse`	Requests e-skin calibration and may change subsequent measurement baseline.
`zero_calibration_status()`	None	`HandServiceResponse`	Queries zero-calibration state/progress without starting it.
`stop_zero_calibration()`	None	`HandServiceResponse`	Requests stop of an active zero calibration; query status afterward.

Tactile and e-skin streams may be high-rate or binary; do not perform expensive decoding or file I/O in callbacks.

SensorClient

Provides IMU, ultrasonic, tactile skin and Lidar data. The SDK automatically decompresses Zstd Lidar data into points; no additional decompression library is required. Use `kind` to select `laser_shots[].points` or `sub_areas[].points`, containing coordinates in meters and reflectivity. Unsupported compression formats

return `kNotSupported`.

```
class SensorClient {
public:
    Result<Subscription> on_imu(std::function<void(const ImuSamples&)> callback,
                                const SubscribeOptions& options = {});
    Result<Subscription> on_ultrasonic(std::function<void(const UltrasonicSamples&)> callback,
                                       const SubscribeOptions& options = {});
    Result<Subscription> on_tactile_skin(std::function<void(const TactileSkinSamples&)> callback,
                                         const SubscribeOptions& options = {});
    Result<Subscription> on_lidar(LidarSource source,
                                  std::function<void(const LidarFrame&)> callback,
                                  const SubscribeOptions& options = {});
    Result<ImuSamples> latest_imu(std::chrono::milliseconds max_age);
    Result<BasicResponse> zero_tactile_skin(bool enabled = true);

};
```

API	Parameters	Callback/result	Purpose and constraints
`on_imu(callback, options)`	Common subscription parameters	`ImuSamples`	Subscribe to one or more IMUs per frame. Angular velocity is rad/s, linear acceleration is m/s², and quaternion fields are w/x/y/z.
`on_ultrasonic(callback, options)`	Common subscription parameters	`UltrasonicSamples`	Subscribe to ultrasonic distance/status. Check both `valid` and `status`; distance is in mm.
`on_tactile_skin(callback, options)`	Common subscription parameters	`TactileSkinSamples`	Subscribe to body tactile-skin force points in N; check each skin's `valid`.
`on_lidar(source, callback, options)`	`source: LidarSource`	`LidarFrame`	Select chassis front, chest front, or chest rear Lidar. Invalid source returns `kInvalidArgument`; Zstd data are automatically decompressed into points.
`latest_imu(max_age)`	Maximum local receive age; Python default 1000 ms	`ImuSamples`	Read fresh IMU cache; missing or expired data returns `kStaleState`.
`zero_tactile_skin(enabled=true)`	`enabled: bool`	`BasicResponse`	`true` uses current force as the 0 N baseline; `false` restores fitted force. This changes interpretation of later readings; check `success/message`.

Sensor availability depends on robot configuration. If a subscription produces no callbacks, check the corresponding data source and connection.

PowerClient

Check sample freshness and reading.quality for power/battery data. Use a reading only when quality is kValid and value is present; zero may be valid. channel_command requires a specific node. Power-changing operations require confirm_power_change; release_estop and shutdown also require a nonempty reason.

```cpp
class PowerClient {
public:
    Result<Subscription> on_chest_state(std::function<void(const PowerBoardState&)> callback,
                                        const SubscribeOptions& options = {});
    Result<Subscription> on_chassis_state(std::function<void(const PowerBoardState&)> callback,
                                          const SubscribeOptions& options = {});
    Result<Subscription> on_battery(std::function<void(const BatterySystemState&)> callback,
                                    const SubscribeOptions& options = {});
    Result<Subscription> on_heartbeat(std::function<void(const Heartbeat&)> callback,
                                      const SubscribeOptions& options = {});
    Result<BatterySystemState> latest_battery(std::chrono::milliseconds max_age);
    Result<PowerCommandResponse> channel_command(const PowerChannelCommand& command);
    Result<PowerCommandResponse> estop(PowerNode node, const std::string& reason);
    Result<PowerCommandResponse> release_estop(PowerNode node, const std::string& reason,
                                               bool confirm_power_change);
    Result<PowerCommandResponse> shutdown(PowerNode node, const std::string& reason,
                                          bool confirm_power_change);

};
```

Power streams may continue publishing when measurements are unavailable. Check sample freshness, `PowerDataQuality`, and optional `value` together; numeric zero may be valid.

API	Parameters	Callback/result	Purpose and constraints
`on_chest_state(callback, options)`	Common subscription parameters	`PowerBoardState`	Subscribe to chest power-board readings, outputs, and switching state.
`on_chassis_state(callback, options)`	Common subscription parameters	`PowerBoardState`	Subscribe to chassis power-board state.
`on_battery(callback, options)`	Common subscription parameters	`BatterySystemState`	Subscribe to voltage, current, SOC, temperature, limits, and fault codes per pack.
`on_heartbeat(callback, options)`	Common subscription parameters	`Heartbeat`	Subscribe to power subsystem lifecycle and faults.
`latest_battery(max_age)`	Maximum local receive age; Python default 1000 ms	`BatterySystemState`	Read fresh battery cache, then validate quality/value for every reading.
`channel_command(command)`	`PowerChannelCommand`	`PowerCommandResponse`	Select a specific node (`kAll` is invalid), opcode, and operation fields. Read-only operations need no confirmation; state-changing opcodes require `confirm_power_change=true`.
`estop(node, reason)`	`PowerNode` and reason	`PowerCommandResponse`	Request emergency stop for a power node. Supply a traceable reason and check `success`, `error_code`, and later state.
`release_estop(node, reason, confirm_power_change=false)`	Node, non-empty reason, explicit confirmation	`PowerCommandResponse`	Releasing estop changes power state. Pass `true`; the default `false` prevents accidental calls.

API	Parameters	Callback/result	Purpose and constraints
`shutdown(node, reason, confirm_power_change=false)`	Node, non-empty reason, explicit confirmation	`PowerCommandResponse`	Shut down a node. Explicit confirmation is mandatory and connection/state streams may stop immediately.

In `PowerChannelCommand`, `channel_id/action` apply to channel operations, `set_mask/clear_mask` to mask operations, and `heartbeat_period_ms` to the corresponding heartbeat-period command; the selected `PowerOpCode` determines the valid combination. Power services use `service_timeout`; after timeout, inspect remaining telemetry or the physical system before retrying.

VideoClient

```
class VideoClient {
public:
    Result<Subscription> on_jpeg(
        VideoSource source,
        std::function<void(const JpegFrame&)> callback,
        const VideoOptions& options = {});
    Result<VideoDiagnostics> diagnostics(VideoSource source) const;
};
```

Subscribe to camera JPEG images and inspect reception diagnostics. Video runtime dependencies are bundled; complete Video environment setup before use.

Method	Parameters	Return	Notes
`on_jpeg(source, callback, options)`	source: head left/right H.265 or wrist left/right H.264; callback: receives JpegFrame; options: VideoOptions	`Result<Subscription>`	Retain the handle. Each camera permits one active subscription, including use by alignment sessions.
`diagnostics(source)`	The subscribed VideoSource	`Result<VideoDiagnostics>`	Inspect received/output counts, drops, resets and the latest error.

`VideoOptions` defaults to NVDEC, JPEG quality 85 and 1280×960 output. Configure `decoder_backend`, `jpeg_quality`, `output_width/output_height` and input queue capacities. Both dimensions set to 0 preserve source size. Select `kSoftware` explicitly for software decoding; there is no automatic fallback, and multi-stream or high-resolution performance is limited.

`JpegFrame` contains the source, header, dimensions and complete JPEG `data`. C++ references are valid only during the callback; copy for asynchronous use. Python data is `bytes`. Keep callbacks short; slow consumers may skip frames.

Invalid parameters return `kInvalidArgument`; an occupied camera returns `kUnavailable`. Inspect returned errors and `diagnostics()` for video environment failures.

AlignmentClient

```
class AlignmentClient {
public:
    Result<Subscription> on_frame(
        const AlignmentOptions& options,
        std::function<void(const AlignedFrame&)> callback,
```

```
        const SubscribeOptions& callback_options = {});
    Result<AlignmentDiagnostics> diagnostics(
        const Subscription& subscription) const;
};
```

Matches samples from multiple sources using timestamps in the same clock domain.

Method	Parameters	Return	Notes
`on_frame(options, callback, callback_options)`	options: AlignmentOptions; callback: receives AlignedFrame	`Result<Subscription>`	Retain the handle. `callback_options` currently does not affect output behavior. Keep callbacks short.
`diagnostics(subscription)`	An active alignment subscription belonging to this SDK	`Result<AlignmentDiagnostics>`	Inspect complete/incomplete frames, timestamp anomalies and per-source matching counts.

AlignmentOptions	Description
`baseline` / `sources`	Baseline and additional sources. Head-left is the default baseline and automatically includes head-right. Other baselines match only explicit sources. Sources must not repeat.
`max_skew` / per-source `max_skew`	Matching tolerance; per-source settings take precedence. 0 requires identical timestamps. Unset values use the defaults below.
Per-source `direction`	Default `kAtOrBefore` selects data no later than the baseline. Alternatives are `kNearest` and `kAtOrAfter`.
`wait_timeout`	Alignment waiting time, default 20 ms.
`buffer_capacity` / `emit_incomplete`	Default per-source capacity 64; incomplete frames are emitted by default.
`video_options`	Shared VideoOptions for video sources, which occupy camera subscriptions.

Default tolerances: arm/torso/chassis/hand joints/IMU 7.5 ms, eskin 15 ms, video 25 ms, finger tactile 50 ms, Master 150 ms. Python tolerance and wait attributes are `max_skew_ns` and `wait_timeout_ms`; `None` uses default tolerance.

`AlignedFrame` contains `timestamp_ns`, `baseline_sample`, `samples` in sources order, and `complete`. Status is Matched, Missing or OutOfTolerance. See Video and alignment data for reading results. `offset_ns` is the candidate timestamp minus the baseline timestamp.

Head JPEG uses RK SEI source time; other devices usually use Header time. Confirm a common clock domain before cross-source alignment. See Example Code Reference for usage.

Time-sync parameters

`SdkConfig.time_sync` uses the fields below. Software mode adjusts host system time; see Time synchronization for host setup.

TimeSyncOptions field	Default and constraints
`enabled`	`true`; disable to skip synchronization.
`mode`	`TimeSyncMode::kSoftware`; alternatively `kHardware`. Python: `TimeSyncMode.SOFTWARE/HARDWARE`.
`interface`	Empty; select the physical wired NIC from the robot endpoint route. Ambiguous selection is an error.
`timeout` / Python `timeout_ms`	60000 ms, strictly positive; initial synchronization preparation deadline.
`failure_policy`	`TimeSyncFailurePolicy::kKeepConnected`: initial synchronization error retains communication. `kFailConnection` closes communication. Python: `KEEP_CONNECTED/FAIL_CONNECTION`.
`software_offset_threshold_ns`	1000000 ns; range 1–1000000000 ns, for software readiness only. Hardware mode uses a fixed 50 ns threshold.

`TimeSyncDiagnostics.state` is `Disabled`, `Preparing`, `Ready`, `Failed` or `Lost` (C++ enum names have a `k` prefix; Python uses uppercase underscores). Readiness requires ten consecutive new observations with absolute offset below the threshold. Check again after synchronization loss during operation.

Diagnostic field	Meaning
`error`	SDK error code and message.
`mode/interface/phc_device`	Selected mode, NIC and hardware clock device; software mode has no PHC device.
`clock_source/system_time_owned`	Clock source, `CLOCK_REALTIME` or `PHC`, and whether this component manages system-time synchronization.
`offset_ns/offset_threshold_ns`	Observed offset and readiness threshold in ns; the threshold determines synchronization readiness.
`stable_samples/required_samples`	Consecutive acceptable observations and required count.
`sample_age` / Python `sample_age_ms`	Diagnostic sample age; over one second is no longer ready.

Data types and fields

The excerpts below cover public types and fields used by the documented APIs, and result-reading methods. Refer to the delivered public headers for declarations. Plan command targets in the application. Inspect quality, heartbeat and freshness before using state data.

Common field meanings

Field	Units and reading rules
`JointState` / `JointElectricalState`	Equal indices identify the same joint. See Joint arrays and coordinates. Read valid fields within the actual array lengths.

Field	Units and reading rules
`ChassisVelocity` / `ChassisState.velocity`	Body +x forward, +y left; positive yaw is counterclockwise viewed from above. `vx/vy` are m/s and `vtheta` is rad/s. Feedback is `velocity.linear.x/y` and `velocity.angular.z`.
`Header.stamp_ns`	Source time; 0 means unavailable. `latest_*` ages data from local reception.
`Header.frame_id` / `Pose` / `Wrench` / `LidarPoint`	`frame_id` identifies a source or coordinate frame. Points are in m, forces in N and torques in Nm. Apply the delivered coordinate definitions before combining devices.
`ImuSamples.imus`	Each entry is one IMU, with rad/s angular velocity and m/s² linear acceleration. Identify sources by their Headers and use the mounting definition when interpreting orientation.
`HandState.force_info`	Raw integer force-sensor information from the device. Use `on_tactile_force()` for fingertip six-axis forces.
`Heartbeat` / `MasterDeviceError`	Heartbeat `expected_mask/online_mask/fault_mask` are device bitmasks. In both types, `fault_codes` are device fault codes. Retain the subsystem and error message for diagnosis.
`BatteryPack`	Pack and cell voltages in V, current in A, `soc/soh` in percent (0–100), power limits in W and temperatures in °C. Check `quality/value` per `PowerReading`.
`PowerBoardState.readings`	Read by `name` and `unit`. Absent `outputs[].enabled` means unknown.
`PowerSampleMetadata`	`clock_domain` identifies the power sample's clock domain; confirm a common clock domain before comparing timestamps.

Configuration data

Type or field	Meaning
`ConfigGetResponse` / `ConfigSetResponse`	`result` is the business outcome and `diagnostics` explains it. Queries return `instances`; writes return an optional `instance`.
`ConfigInstanceInfo` / `ConfigFieldInfo`	`plugin_id` identifies the instance, `field_id` the field, and `field_descriptor` its type, unit, constraints and safety conditions.
`ConfigFieldOperation`	`field_id`, `operation` and optional `value`. `ConfigValue` supports bool, int64, finite double, string, double array and int64 array. Python constructors are `boolean/integer/real/string/double_array/integer_array()`.
`effective_value` / `saved_override`	Current effective value and saved override; either may be absent.
`apply_policy` / `pending_init`	Application policy is IMMEDIATE, SAFE_STATE or INIT_ONLY. `pending_init` indicates a change awaiting initialization.
`ConfigDiagnostic`	`code/field_id/message` identify the problem; optional `element_index` identifies an array element.

On `RecoveryRequired`, restore device state before changing configuration.

Video and alignment data

Type	Essential fields and rules
`VideoSource`	`kHeadLeft/kHeadRight/kWristLeft/kWristRight`; Python: `HEAD_LEFT/HEAD_RIGHT/WRIST_LEFT/WRIST_RIGHT`.
`VideoOptions`	`jpeg_quality` defaults to 85 (1–100); `decoder_backend` defaults to NVDEC. Default dimensions are 1280×960; both zero preserve input size. `max_pending_frames` defaults to 32 (1–4096), `max_pending_bytes` to 16 MiB, and `max_au_bytes` to 4 MiB (positive, at most 64 MiB and no larger than `max_pending_bytes`). Width and height must not exceed 8192.
`JpegFrame`	`source/header/width/height/data` describe a complete JPEG. Optional `source_timestamp_ns` requires checking `timestamp_status` (Valid/Missing/Invalid/NotApplicable). `sdk_latency_ns` covers input reception to C++ callback entry; `pipeline_latency_ns` covers processing inside the decoder pipeline. Neither includes browser display or Python GIL wait.
`VideoDiagnostics`	`active/waiting_for_keyframe`, `received_au_count/jpeg_frame_count`, drop/reset counters and `last_error` distinguish no input, waiting for a keyframe and decode failure.
`AlignmentSource`	`kind` selects the type. Arms/hands use `side`; tactile sources also use `finger/tactile_image_kind`; video uses `video_source`. Match controls are `direction` and optional `max_skew`; Python tolerance is `max_skew_ns`.
`AlignmentDataKind`	Supports ChassisState, TorsoState, ArmState, HandState, TactileForce, TactileImage, HandEskinForce, HandEskinRaw, ImuSamples, TactileSkinSamples, VideoJpeg and MasterStatus. C++ uses a k prefix; Python uses uppercase underscores.
`AlignedSample`	`source/status/timestamp_ns/offset_ns/value`. When `matched()` (Python property `matched`) is true, read the matched result with C++ `get<T>()` or Python `value`.
`AlignedFrame`	`timestamp_ns/baseline_sample` are the baseline time and data. `samples` follows explicit `sources` order. `complete` means all required sources matched. Default stereo also exposes left/right results through `left_image/right_sample`.
`AlignmentDiagnostics`	Frame counts: `evaluated_frame_count/emitted_frame_count/incomplete_frame_count`. Per-source `baseline/sources` expose `received_count/matched_count/missing_count/out_of_tolerance_count` to diagnose missing input or tolerance failures. `invalid_timestamp_count/regression_count` identify invalid or regressing time.

Before computing data latency from `robot_time_ns()`, confirm that the source uses the same gPTP domain. For head video, use `source_timestamp_ns` with a valid `timestamp_status`.

Basic type declarations

```
enum class ErrorCode {
    kOk = 0,
    kInvalidArgument,
    kNotConnected,
    kUnavailable,
    kTimeout,
    kSerialization,
    kTransport,
    kStaleState,
    kOutcomeUnknown,
    kRemoteFailure,
    kInternal,
    kMotionNotAllowed,
    kNotSupported,
    kTimeSyncUnavailable,
    kTimeSyncPermission,
    kTimeSyncUnsupported,
    kTimeSyncConflict,
    kTimeSyncTimeout,
    kTimeSyncLost,
};

struct Error {
    ErrorCode code = ErrorCode::kOk;
    std::string message;
};

class Status {
public:
    bool ok() const noexcept;
    explicit operator bool() const noexcept;
    ErrorCode code() const noexcept;
    const std::string& message() const noexcept;
};

template <typename T>
class Result {
public:
    bool ok() const noexcept;
    explicit operator bool() const noexcept;
    T& value() &;
    const T& value() const&;
    T&& value() &&;
    const Error& error() const noexcept;
};

enum class Side { kLeft = 1, kRight = 2 };
enum class ControlMode {
    kJointPosition = 0,
    kJointPd = 1,
    kJointTorque = 2,
    kSpeed = 3,
    kFloating = 4,
    kMotorPd = 5,
    kCartesianImpedance = 6,
};
enum class LifecyclePhase { kUnspecified = 0, kInit, kStandby, kRunning, kError, kFatal };
enum class OperationMode { kUnspecified = 0, kTeleoperation, kInference, kAutomation };
enum class MasterPhase { kUnspecified = 0, kInit, kStandby, kRunning, kError, kOta, kEstop };
enum class CommandChannel {
    kUnspecified = 0,
    kTeleoperation = 1,
    kInference = 2,
    kAutomation = 3,
    kUniversal = 4,
};
enum class TorsoControlMode { kPosition = 0, kMit = 1 };
enum class HandMode {
    kLegacyPosition = 0,
    kLegacyZeroForce = 1,
    kZeroForce = 10,
    kPosition = 13,
    kAdmittance = 14,
    kMit = 15,
```

```
};
enum class HealthRequirement { kUnspecified = 0, kOk, kWarning, kOff };
enum class HardwareHealthVerdict { kOk = 0, kWarning, kFault, kStale, kNeverSeen };
enum class MasterResult {
    kUnspecified = 0,
    kOk,
    kInvalidRequest,
    kInvalidPhase,
    kUnknownTarget,
    kBusy,
    kUnavailable,
    kHealthCheckFailed,
    kExecutionFailed,
    kOutcomeUnknown,
    kInternalError,
};

struct Header {
    std::int64_t stamp_ns = 0;
    std::int64_t system_time_ns = 0;
    std::uint32_t seq = 0;
    std::string key;
    std::string frame_id;
};

struct Vector3 {
    float x = 0, y = 0, z = 0;
};
struct Quaternion {
    float w = 1, x = 0, y = 0, z = 0;
};
struct Pose {
    Vector3 position;
    Quaternion orientation;
};
struct Twist {
    Vector3 linear;
    Vector3 angular;
};
struct Wrench {
    Vector3 force;
    Vector3 torque;
};

struct JointState {
    std::vector<std::string> name;
    std::vector<float> position;
    std::vector<float> velocity;
    std::vector<float> torque;
};

struct JointCommand {
    std::vector<std::string> name;
    std::vector<float> position;
    std::vector<float> velocity;
    std::vector<float> torque;
};

struct JointElectricalState {
    std::vector<std::string> name;
    std::vector<float> temperature_celsius;
    std::vector<float> current;
    std::vector<float> bus_voltage;
};

struct HeartbeatMetric {
    std::uint32_t kind = 0;
    std::uint32_t instance = 0;
    float value = 0;
};

struct Heartbeat {
    Header header;
    std::string subsystem;
    std::uint32_t seq = 0;
    LifecyclePhase phase = LifecyclePhase::kUnspecified;
    std::uint32_t severity = 0;
    ControlMode control_mode = ControlMode::kJointPosition;
    std::uint32_t expected_mask = 0;
```

```
    std::uint32_t online_mask = 0;
    std::uint32_t fault_mask = 0;
    std::vector<std::uint32_t> fault_codes;
    std::uint32_t error_code = 0;
    float feedback_age_ms = 0;
    float control_rate_hz = 0;
    float state_rate_hz = 0;
    float publish_rate_hz = 0;
    float bus_load_pct = 0;
    std::vector<HeartbeatMetric> metrics;
};

struct MasterStatus {
    Header header;
    MasterPhase phase = MasterPhase::kUnspecified;
    OperationMode mode = OperationMode::kUnspecified;
    std::string task_id;
    bool degraded = false;
};

struct HardwareHealth {
    Heartbeat heartbeat;
    std::uint64_t receive_age_ms = 0;
    HardwareHealthVerdict verdict = HardwareHealthVerdict::kNeverSeen;
    HealthRequirement requirement = HealthRequirement::kUnspecified;
};

struct SafetyGuardStatus {
    std::string guard_id;
    bool enabled = false;
};

struct HostResourceMetrics {
    float cpu_temperature_celsius = 0;
    float cpu_load_pct = 0;
    float memory_used_pct = 0;
    std::vector<float> cpu_core_load_pct;
};

struct CommandChannelRate {
    CommandChannel channel = CommandChannel::kUnspecified;
    float receive_hz = 0;
    float accepted_hz = 0;
};

struct SystemHardwareHealth {
    Header header;
    std::uint32_t worst_severity = 0;
    std::vector<HardwareHealth> subsystems;
    MasterStatus master_status;
    std::vector<SafetyGuardStatus> safety_guards;
    HostResourceMetrics host;
    std::vector<CommandChannelRate> command_rates;
};

struct MasterDeviceError {
    std::string subsystem_id;
    HealthRequirement required = HealthRequirement::kUnspecified;
    HardwareHealthVerdict actual = HardwareHealthVerdict::kNeverSeen;
    std::uint32_t error_code = 0;
    std::uint32_t fault_mask = 0;
    std::vector<std::uint32_t> fault_codes;
};

struct MasterEvent {
    Header header;
    std::string task_id;
    bool accepted = false;
    bool active = false;
    std::int32_t status = 0;
    std::uint32_t error_code = 0;
    std::string reason;
    std::string guard_id;
    std::vector<MasterDeviceError> device_errors;
};

struct ArmState {
    Header header;
    Side side = Side::kLeft;
```

```
    Pose cartesian;
    JointState joints;
    Wrench wrench;
    JointElectricalState electrical;
};

struct ArmCommand {
    JointCommand joints;
};

struct HandForceInfo {
    std::uint32_t online = 0;
    std::array<std::int32_t, 4> strain_sensors{};
    std::int32_t torque_x = 0;
    std::int32_t torque_y = 0;
    std::int32_t force_z = 0;
    std::int32_t temperature_celsius = 0;
    std::int32_t error_code = 0;
};

struct HandState {
    Header header;
    Side side = Side::kLeft;
    JointState joints;
    JointElectricalState electrical;
    std::vector<HandForceInfo> force_info;
};

struct HandCommand {
    JointCommand joints;
};

struct TorsoState {
    Header header;
    JointState joints;
    JointElectricalState electrical;
};

struct TorsoCommand {
    JointCommand joints;
};

struct ChassisState {
    Header header;
    Twist velocity;
    JointState joints;
    JointElectricalState electrical;
    std::vector<std::uint32_t> joint_mode;
};

struct ChassisVelocity {
    float vx = 0;
    float vy = 0;
    float vtheta = 0;
    float extension_velocity = 0;
};

enum class ChassisControlMode { kMotion = 0, kForce = 1 };

struct ImuSample {
    Header header;
    Quaternion orientation;
    Vector3 angular_velocity;
    Vector3 linear_acceleration;
    Quaternion aligned_orientation;
    Vector3 gravity;
};

struct ImuSamples {
    std::uint64_t stamp_ns = 0;
    std::vector<ImuSample> imus;
};

struct WholeBodyCommand {
    std::optional<ArmCommand> left_arm;
    std::optional<ArmCommand> right_arm;
    std::optional<HandCommand> left_hand;
    std::optional<HandCommand> right_hand;
    std::optional<TorsoCommand> torso;
```

```
    std::optional<ChassisVelocity> chassis;
};

struct BasicResponse {
    bool success = false;
    std::string message;
};

struct ModeResponse : BasicResponse {
    std::uint32_t previous_mode = 0;
    std::uint32_t current_mode = 0;
};

struct ChassisModeResponse : BasicResponse {
    ChassisControlMode previous_mode = ChassisControlMode::kMotion;
    ChassisControlMode current_mode = ChassisControlMode::kMotion;
};

enum class MasterResponseDetailKind {
    kNone = 0,
    kStartTask,
    kStopTask,
    kSetMode,
    kHealthRequirement,
    kSafetyGuard,
    kPhaseTransition,
};

struct MasterResponseDetail {
    MasterResponseDetailKind kind = MasterResponseDetailKind::kNone;
    std::string task_id;
    std::int32_t status = 0;
    OperationMode requested_mode = OperationMode::kUnspecified;
    std::string subsystem_id;
    HealthRequirement requirement = HealthRequirement::kUnspecified;
    std::string guard_id;
    bool enabled = false;
    std::uint32_t phase_transition_operation = 0;
};

struct MasterResponse {
    Header header;
    MasterResult result = MasterResult::kUnspecified;
    std::string message;
    MasterStatus before;
    MasterStatus after;
    MasterResponseDetail detail;
};

struct HandServiceResponse : BasicResponse {
    std::uint32_t result = 0;
    std::uint32_t previous_mode = 0;
    std::uint32_t current_mode = 0;
    std::int32_t sdk_error_code = 0;
    std::uint32_t calibration_state = 0;
    std::uint32_t progress_percent = 0;
};

struct CompressedImage {
    Header header;
    std::string format;
    std::string data;
};

struct TactileForce {
    Header header;
    std::uint32_t finger = 0;
    Wrench wrench;
};

enum class TactileImageKind { kRaw = 0, kDeform = 1 };

struct HandEskinForce {
    Header header;
    std::int32_t frame_id = 0;
    std::int32_t channel = 0;
    std::vector<float> force_n;
};
```

```cpp
struct HandEskinRaw {
    Header header;
    std::int32_t frame_id = 0;
    std::int32_t channel = 0;
    std::string data;
};

enum class UltrasonicStatus {
    kUnspecified = 0,
    kValid,
    kNoTarget,
    kInterference,
    kNotReady,
    kInvalid,
};

struct UltrasonicSample {
    Header header;
    std::uint32_t distance_mm = 0;
    std::uint32_t raw_value = 0;
    float temperature_celsius = 0;
    UltrasonicStatus status = UltrasonicStatus::kUnspecified;
    bool valid = false;
};

struct UltrasonicSamples {
    std::uint64_t stamp_ns = 0;
    std::vector<UltrasonicSample> samples;
};

struct TactileSkinPoint {
    std::string serial_id;
    std::uint32_t row = 0;
    std::uint32_t col = 0;
    float force_n = 0;
};

struct TactileSkin {
    Header header;
    std::vector<TactileSkinPoint> points;
    bool valid = false;
};

struct TactileSkinSamples {
    std::uint64_t stamp_ns = 0;
    std::vector<TactileSkin> skins;
};

enum class LidarSource { kChassisFront = 0, kChestFront, kChestRear };

enum class LidarPayloadKind {
    kNone = 0,
    kLaserShots,
    kSubAreas,
    kCompressedLaserShots,
    kCompressedSubAreas,
};

struct LidarPoint {
    float x = 0, y = 0, z = 0;
    std::uint32_t reflectivity = 0;
};

struct LidarLaserShot {
    std::uint64_t timestamp_ns = 0;
    float azimuth_rad = 0;
    std::vector<LidarPoint> points;
};

struct LidarSubArea {
    std::uint64_t timestamp_ns = 0;
    std::uint32_t row_id = 0;
    std::uint32_t column_id = 0;
    std::vector<LidarPoint> points;
};

struct LidarFrame {
    Header header;
    std::uint64_t stamp_ns = 0;
```

```
    LidarPayloadKind kind = LidarPayloadKind::kNone;
    std::vector<LidarLaserShot> laser_shots;
    std::vector<LidarSubArea> sub_areas;
    std::string compression;
    std::uint32_t packed_group_count = 0;
    std::uint32_t packed_point_count = 0;
    std::string packed_groups;
    std::string packed_points;
};

enum class PowerDataQuality {
    kUnspecified = 0,
    kValid,
    kNotReceived,
    kStale,
    kSourceOffline,
    kSensorInvalid,
    kNotSupported,
    kUnverified,
};
enum class PowerLinkState { kUnknown = 0, kOnline, kOffline };
enum class PowerClockDomain { kUnknown = 0, kDeviceMonotonic, kGptp };

struct PowerReading {
    std::optional<float> value;
    PowerDataQuality quality = PowerDataQuality::kUnspecified;
    std::uint64_t sample_time_ns = 0;
};

struct NamedPowerReading {
    std::string name;
    std::string unit;
    PowerReading reading;
};

struct PowerSampleMetadata {
    bool received = false;
    bool transport_online = false;
    PowerLinkState source_link = PowerLinkState::kUnknown;
    std::int64_t received_at_ns = 0;
    std::uint64_t receive_age_ms = 0;
    std::uint64_t sample_time_ns = 0;
    std::uint64_t envelope_time_ns = 0;
    PowerClockDomain clock_domain = PowerClockDomain::kUnknown;
    std::uint32_t envelope_seq = 0;
    std::uint32_t sample_seq = 0;
};

struct PowerOutput {
    std::string name;
    std::optional<bool> enabled;
};

struct PowerBoardState {
    Header header;
    PowerSampleMetadata sample;
    std::vector<NamedPowerReading> readings;
    std::uint32_t operating_state = 0;
    std::vector<PowerOutput> outputs;
    std::optional<bool> switching;
};

struct BatteryPack {
    std::uint32_t pack_id = 0;
    PowerLinkState online = PowerLinkState::kUnknown;
    PowerDataQuality quality = PowerDataQuality::kUnspecified;
    PowerReading voltage;
    PowerReading current;
    PowerReading soc;
    PowerReading soh;
    PowerReading minimum_cell_voltage;
    PowerReading maximum_cell_voltage;
    PowerReading charge_power_limit;
    PowerReading discharge_power_limit;
    PowerReading ambient_temperature;
    std::vector<PowerReading> cell_temperatures;
    std::vector<PowerReading> mos_temperatures;
    std::optional<std::uint32_t> fault_code;
    std::optional<std::uint32_t> alarm_code;
```

```
    std::optional<std::uint32_t> protection_code;
};

struct BatterySystemState {
    Header header;
    PowerSampleMetadata sample;
    std::vector<BatteryPack> packs;
};

enum class PowerNode : std::uint32_t {
    kAll = 0x00,
    kChassis = 0x10,
    kBody = 0x20,
    kRt1186 = 0x30,
};

enum class PowerOpCode : std::uint32_t {
    kNop = 0,
    kPing = 1,
    kGetProtocolVersion = 2,
    kSetChannel = 16,
    kSetMask = 17,
    kGetStatus = 18,
    kAllOn = 19,
    kAllOff = 20,
    kSystemSleep = 32,
    kSystemWakeup = 33,
    kSystemGetStatus = 34,
    kSystemReset = 35,
    kSystemSetHeartbeatPeriod = 36,
};

struct PowerChannelCommand {
    PowerNode node = PowerNode::kBody;
    PowerOpCode opcode = PowerOpCode::kNop;
    std::uint32_t channel_id = 0;
    std::uint32_t action = 0;
    std::uint32_t set_mask = 0;
    std::uint32_t clear_mask = 0;
    std::uint32_t heartbeat_period_ms = 0;
    bool confirm_power_change = false;
};

struct PowerCommandResponse : BasicResponse {
    std::uint32_t node = 0;
    std::uint32_t error_code = 0;
    std::uint32_t state_mask = 0;
};

enum class QueuePolicy {
    kDropOldest = 0,
    kLatestOnly,
    kReliable,
};

struct SubscribeOptions {
    QueuePolicy queue_policy = QueuePolicy::kDropOldest;
};

struct SdkConfig {
    std::string zenoh_config_path;
    std::vector<std::string> connect_endpoints;
    std::chrono::milliseconds connect_timeout{5000};
    std::chrono::milliseconds service_timeout{6000};
    std::chrono::milliseconds master_timeout{15000};
    std::chrono::milliseconds torso_timeout{20000};
    std::size_t callback_queue_capacity = 256;
    std::chrono::milliseconds master_status_max_age{500};
    CommandChannel command_channel = CommandChannel::kUnspecified;
    TimeSyncOptions time_sync;
};

struct CallbackDiagnostics {
    std::uint64_t dropped_callback_count = 0;
    std::uint64_t coalesced_callback_count = 0;
    Error last_decode_error;
};
```

Lifecycle and diagnostics

Rebuild user subscriptions after reconnect. `dropped_callback_count` and `coalesced_callback_count` count cumulative dropped and coalesced callbacks; `last_decode_error` is the latest decode error.

Example Code Reference

Example index

Examples are organized by common tasks. Start with quickstart, then select a Client example. Motion examples have separate prerequisites. Combine one example function with the shared entry point to create a standalone application.

Client	Task	C++ call	Python call
MasterClient	Subscribe to Master status	`master_status(*sdk);`	`master_status(sdk)`
ArmClient	Read cached left-arm state	`arm_cache(*sdk);`	`arm_cache(sdk)`
TorsoClient	Subscribe to torso state	`torso_state(*sdk);`	`torso_state(sdk)`
HandClient	Subscribe to hand tactile images	`hand_tactile(*sdk);`	`hand_tactile(sdk)`
HandClient	Query hand calibration status	`hand_calibration_status(*sdk);`	`hand_calibration_status(sdk)`
SensorClient	Subscribe to IMU	`imu_state(*sdk);`	`imu_state(sdk)`
SensorClient	Subscribe to Lidar points	`lidar_points(*sdk);`	`lidar_points(sdk)`
PowerClient	Read valid battery measurements	`battery_state(*sdk);`	`battery_state(sdk)`
VideoClient	Subscribe to head JPEG	`head_jpeg(*sdk);`	`head_jpeg(sdk)`
AlignmentClient	Align two-arm state	`aligned_arms(*sdk);`	`aligned_arms(sdk)`
ConfigClient	Query configuration instances	`config_instances(*sdk);`	`config_instances(sdk)`
MasterClient	Master mode service	`change_master_mode(*sdk, requested);`	`change_master_mode(sdk, requested)`
ArmClient	Send a single-arm target	`send_right_arm(*sdk, target);`	`send_right_arm(sdk, target)`
WholeBodyClient	WholeBody two-arm command	`send_two_arms(*sdk, channel, left, right);`	`send_two_arms(sdk, channel, left, right)`

Calls using `requested`, `target`, `channel`, `left` or `right` require application-provided arguments and the mode-change or motion prerequisites stated in their sections.

Included quickstart programs

`examples/cpp/quickstart.cpp` and `examples/python/quickstart.py` demonstrate left-arm state subscription, handle retention and clean shutdown. Both accept an endpoint on the command line. C++ also accepts a Zenoh configuration path; Python uses `--zenoh-config`. State callbacks print sequence and joint

count; press Ctrl+C to exit.

The example selects the left arm with `sdk.arm(Side::kLeft)` / `sdk.arm(Side.LEFT)`. Omitting `SubscribeOptions` uses the default DropOldest callback policy. Retain the Subscription until shutdown. See `ArmClient::on_state` for complete parameters and queue semantics.

```
"$NORTHSDK_PREFIX/bin/north_sdk_quickstart" "$NORTHSDK_ENDPOINT"
python3 "$NORTHSDK_PREFIX/examples/python/quickstart.py" --endpoint "$NORTHSDK_ENDPOINT"
```

Load the environment and set the robot endpoint through Quick Start. Commands run from any directory.

Read-only connection diagnostics

The installed north_sdk_robot_smoke counts state samples and rates over a bounded duration to check connection and reception. The complete package also includes the Python equivalent:

```
"$NORTHSDK_PREFIX/bin/north_sdk_robot_smoke" --endpoint "$NORTHSDK_ENDPOINT" --seconds 5
python3 "$NORTHSDK_PREFIX/examples/python/robot_smoke.py" --endpoint "$NORTHSDK_ENDPOINT" --seconds 5
```

Receiving Master status, hardware health and at least one heartbeat produces PASS and exit code 0. Check silent streams against the actual device configuration. Add `--query` for read-only service queries and inspect each result.

Shared entry point

C++: create main.cpp containing these includes, one selected example function, then the entry point, in that order. Replace the default master_status(*sdk) call for other examples. Integrate the selected example into your existing C++ application using the delivered public headers and libraries.

```
#include <north_sdk/NorthSDK.h>
#include <array>
#include <chrono>
#include <iostream>
#include <stdexcept>
#include <thread>
#include <utility>
```

```
int main(int argc, char** argv) {
    if (argc != 2) {
        std::cerr << "usage: north_sdk_app <endpoint>\n";
        return 2;
    }
    try {
        north::SdkConfig config;
        config.connect_endpoints = {argv[1]};
        auto created = north::NorthSDK::Create(config);
        if (!created) throw std::runtime_error(created.error().message);
        auto sdk = std::move(created.value());
        auto connected = sdk->connect();
        if (!connected) throw std::runtime_error(connected.message());
        master_status(*sdk);
        sdk->close();
    } catch (const std::exception& error) {
        std::cerr << error.what() << '\n';
        return 1;
```

```
    }
    return 0;
}
```

Compile and run using `NORTHSDK_PREFIX` from Quick Start:

```
g++ -std=c++17 main.cpp \
  -I"$NORTHSDK_PREFIX/include" -L"$NORTHSDK_PREFIX/lib" -lnorth_sdk \
  -Wl,-rpath,"$NORTHSDK_PREFIX/lib" \
  -o north_sdk_app
./north_sdk_app "$NORTHSDK_ENDPOINT"
```

Python: create app.py with these imports, one selected example function, then the entry point. Replace the example call in the entry point.

```
import sys
import time
from north_sdk import (
    NorthSDK, SdkConfig, NorthSDKError, ErrorCode, Side,
    TactileImageKind, PowerDataQuality, MasterResult, LidarSource, LidarPayloadKind,
    OperationMode, CommandChannel, WholeBodyCommand, ArmCommand,
)
```

```
def main():
    if len(sys.argv) != 2:
        raise SystemExit("usage: python3 app.py <endpoint>")
    config = SdkConfig()
    config.connect_endpoints = [sys.argv[1]]
    try:
        with NorthSDK(config) as sdk:
            master_status(sdk)  # Replace with one example call from the index.
    except (NorthSDKError, RuntimeError) as error:
        print(error, file=sys.stderr)
        raise SystemExit(1)

if __name__ == "__main__":
    main()
```

```
python3 app.py udp/192.168.100.100:7449
```

The read-only subscription examples close their subscriptions after five seconds and require the robot to publish the corresponding data. The entry point reports exceptions, while each example checks business results.

Subscribe to Master status

Receive robot phase, mode and task ID. Retain the subscription for five seconds; an active publisher produces multiple lines.

C++

```
void master_status(north::NorthSDK& sdk) {
    auto result = sdk.master().on_status([](const north::MasterStatus& state) {
        std::cout << "phase=" << static_cast<int>(state.phase)
                  << " mode=" << static_cast<int>(state.mode)
                  << " task=" << state.task_id << '\n';
    });
    if (!result) throw std::runtime_error(result.error().message);
    auto sub = std::move(result.value());
    std::this_thread::sleep_for(std::chrono::seconds(5));
```

```
    sub.close();
}
```

Python

```
def master_status(sdk):
    sub = sdk.master().on_status(
        lambda state: print(state.phase, state.mode, state.task_id)
    )
    try:
        time.sleep(5)
    finally:
        sub.close()
```

Read cached left-arm state

Try for up to two seconds to read left-arm state received within 500 ms. The first call may lack data. Retry only kStaleState; valid state prints the position count. Cache reads establish the required state reception automatically.

This example uses a 500 ms cache-age limit and a two-second first-sample wait. Choose values from the device publication period and application latency requirements. See `ArmClient::latest_state`.

C++

```
void arm_cache(north::NorthSDK& sdk) {
    auto arm = sdk.arm(north::Side::kLeft);
    const auto deadline = std::chrono::steady_clock::now() + std::chrono::seconds(2);
    while (std::chrono::steady_clock::now() < deadline) {
        auto result = arm.latest_state(std::chrono::milliseconds(500));
        if (result) {
            std::cout << "joints=" << result.value().joints.position.size() << '\n';
            return;
        }
        if (result.error().code != north::ErrorCode::kStaleState)
            throw std::runtime_error(result.error().message);
        std::this_thread::sleep_for(std::chrono::milliseconds(50));
    }
    throw std::runtime_error("No fresh left-arm state within 2 seconds");
}
```

Python

```
def arm_cache(sdk):
    arm = sdk.arm(Side.LEFT)
    deadline = time.monotonic() + 2
    while time.monotonic() < deadline:
        try:
            state = arm.latest_state(500)
        except NorthSDKError as error:
            if error.code != ErrorCode.STALE_STATE:
                raise
            time.sleep(0.05)
        else:
            print("joints=", len(state.joints.position))
            return
    raise RuntimeError("No fresh left-arm state within 2 seconds")
```

Subscribe to torso state

Retain a subscription for five seconds and print position count. Complete joint positions contain eight entries: body J1–J5 then neck J6–J8.

C++

```cpp
void torso_state(north::NorthSDK& sdk) {
    auto result = sdk.torso().on_state([](const north::TorsoState& state) {
        std::cout << "torso joints=" << state.joints.position.size() << '\n';
    });
    if (!result) throw std::runtime_error(result.error().message);
    auto sub = std::move(result.value());
    std::this_thread::sleep_for(std::chrono::seconds(5));
    sub.close();
}
```

Python

```python
def torso_state(sdk):
    sub = sdk.torso().on_state(
        lambda state: print("torso joints=", len(state.joints.position))
    )
    try:
        time.sleep(5)
    finally:
        sub.close()
```

Subscribe to hand tactile images

Subscribe to the left thumb raw tactile image for five seconds. finger=4 selects the thumb. Print format and binary payload length; use the message format when decoding.

C++

```cpp
void hand_tactile(north::NorthSDK& sdk) {
    auto result = sdk.hand(north::Side::kLeft).on_tactile_image(
        4, north::TactileImageKind::kRaw, [](const north::CompressedImage& image) {
            std::cout << "format=" << image.format
                      << " bytes=" << image.data.size() << '\n';
        });
    if (!result) throw std::runtime_error(result.error().message);
    auto sub = std::move(result.value());
    std::this_thread::sleep_for(std::chrono::seconds(5));
    sub.close();
}
```

Python

```python
def hand_tactile(sdk):
    sub = sdk.hand(Side.LEFT).on_tactile_image(
        4, TactileImageKind.RAW,
        lambda image: print("format=", image.format, "bytes=", len(image.data)),
    )
    try:
        time.sleep(5)
```

```
    finally:
        sub.close()
```

Query hand calibration status

Query left-hand zero-calibration status. Handle SDK failure and business rejection separately, then print the response message.

C++

```cpp
void hand_calibration_status(north::NorthSDK& sdk) {
    auto result = sdk.hand(north::Side::kLeft).zero_calibration_status();
    if (!result) throw std::runtime_error("SDK error: " + result.error().message);
    if (!result.value().success)
        throw std::runtime_error("Service rejected: " + result.value().message);
    std::cout << result.value().message << '\n';
}
```

Python

```python
def hand_calibration_status(sdk):
    response = sdk.hand(Side.LEFT).zero_calibration_status()
    if not response.success:
        raise RuntimeError("Service rejected: " + response.message)
    print(response.message)
```

Subscribe to IMU

Subscribe for five seconds. Iterate through imus in each sample and print angular velocity x in rad/s. The robot must provide an IMU source.

C++

```cpp
void imu_state(north::NorthSDK& sdk) {
    auto result = sdk.sensors().on_imu([](const north::ImuSamples& samples) {
        for (const auto& imu : samples.imus)
            std::cout << "angular_velocity.x=" << imu.angular_velocity.x << '\n';
    });
    if (!result) throw std::runtime_error(result.error().message);
    auto sub = std::move(result.value());
    std::this_thread::sleep_for(std::chrono::seconds(5));
    sub.close();
}
```

Python

```python
def imu_state(sdk):
    def display(samples):
        for imu in samples.imus:
            print("angular_velocity.x=", imu.angular_velocity.x)
    sub = sdk.sensors().on_imu(display)
    try:
        time.sleep(5)
```

```
    finally:
        sub.close()
```

Subscribe to Lidar points

Subscribe to the chassis-front Lidar for five seconds and print the first point of each frame: coordinates in meters and reflectivity. The SDK decompresses Zstd automatically. Select chest front or chest rear as needed; the robot must have the corresponding device installed and publishing.

C++

```
void lidar_points(north::NorthSDK& sdk) {
    auto result = sdk.sensors().on_lidar(
        north::LidarSource::kChassisFront, [](const north::LidarFrame& frame) {
            auto print_first = [](const auto& groups) {
                for (const auto& group : groups) {
                    if (group.points.empty()) continue;
                    const auto& point = group.points.front();
                    std::cout << "xyz(m)=" << point.x << ',' << point.y << ',' << point.z
                              << " reflectivity=" << point.reflectivity << '\n';
                    break;
                }
            };
            if (frame.kind == north::LidarPayloadKind::kLaserShots)
                print_first(frame.laser_shots);
            else if (frame.kind == north::LidarPayloadKind::kSubAreas)
                print_first(frame.sub_areas);
        });
    if (!result) throw std::runtime_error(result.error().message);
    auto sub = std::move(result.value());
    std::this_thread::sleep_for(std::chrono::seconds(5));
    sub.close();
}
```

Python

```
def lidar_points(sdk):
    def display(frame):
        if frame.kind == LidarPayloadKind.LASER_SHOTS:
            groups = frame.laser_shots
        elif frame.kind == LidarPayloadKind.SUB_AREAS:
            groups = frame.sub_areas
        else:
            return
        for group in groups:
            points = group.points
            if points:
                point = points[0]
                print("xyz(m)=", point.x, point.y, point.z,
                      "reflectivity=", point.reflectivity)
                break
    sub = sdk.sensors().on_lidar(LidarSource.CHASSIS_FRONT, display)
    try:
        time.sleep(5)
    finally:
        sub.close()
```

Read valid battery measurements

Subscribe to battery state for five seconds. Check voltage.quality and optional value for each pack, printing voltage in V only when valid. Otherwise print unavailable. If callbacks are absent, inspect the source and connection.

C++

```cpp
void battery_state(north::NorthSDK& sdk) {
    auto result = sdk.power().on_battery([](const north::BatterySystemState& state) {
        for (const auto& pack : state.packs) {
            std::cout << "pack=" << pack.pack_id << " voltage=";
            if (pack.voltage.quality == north::PowerDataQuality::kValid
                    && pack.voltage.value)
                std::cout << *pack.voltage.value << " V\n";
            else
                std::cout << "unavailable\n";
        }
    });
    if (!result) throw std::runtime_error(result.error().message);
    auto sub = std::move(result.value());
    std::this_thread::sleep_for(std::chrono::seconds(5));
    sub.close();
}
```

Python

```python
def battery_state(sdk):
    def display(state):
        for pack in state.packs:
            reading = pack.voltage
            if reading.quality == PowerDataQuality.VALID and reading.value is not None:
                print("pack=", pack.pack_id, "voltage=", reading.value, "V")
            else:
                print("pack=", pack.pack_id, "voltage=unavailable")
    sub = sdk.power().on_battery(display)
    try:
        time.sleep(5)
    finally:
        sub.close()
```

Subscribe to head JPEG

This example selects head-left with NVIDIA hardware decoding by default. Load the package environment and complete the video environment check first.

Software-decoder options for each program are listed below:

Program or API	Select software decoding
C++ capture program `north_sdk_video_jpeg`	Pass `--software` at runtime; `--nvdec` selects hardware
Four-camera preview `examples/python/four_camera_viewer.py`	Pass `--decoder software`; `--decoder nvdec` selects hardware
Your C++ application	Before subscribing, set `options.decoder_backend = north::VideoDecoderBackend::kSoftware`

Program or API	Select software decoding
Your Python application	Before subscribing, set `options.decoder_backend = VideoDecoderBackend.SOFTWARE`

The minimal subscription example below retains default hardware decoding; use the table to select software.

Browser preview

Preview both head and both wrist cameras:

```
python3 "$NORTHSDK_PREFIX/examples/python/four_camera_viewer.py" \
  --endpoint "$NORTHSDK_ENDPOINT" --decoder nvdec --port 8080
```

The program opens a local browser; alternatively visit `http://127.0.0.1:8080/`. Without an NVIDIA GPU, use `--decoder software`. Press Ctrl+C or click the stop-preview button to exit. Cameras must already be online and publishing.

C++

```cpp
void head_jpeg(north::NorthSDK& sdk) {
    north::VideoOptions options;
    auto result = sdk.video().on_jpeg(
        north::VideoSource::kHeadLeft,
        [](const north::JpegFrame& frame) {
            std::cout << frame.width << 'x' << frame.height
                      << " bytes=" << frame.data.size() << '\n';
        }, options);
    if (!result) throw std::runtime_error(result.error().message);
    auto subscription = std::move(result.value());
    std::this_thread::sleep_for(std::chrono::seconds(5));
    subscription.close();
}
```

Python

```python
def head_jpeg(sdk):
    from north_sdk import VideoDecoderBackend, VideoOptions, VideoSource

    options = VideoOptions()
    subscription = sdk.video().on_jpeg(
        VideoSource.HEAD_LEFT,
        lambda frame: print(frame.width, frame.height, len(frame.data)),
        options,
    )
    time.sleep(5)
    subscription.close()
```

Only one active subscription is allowed per source; retain the handle. See `VideoClient` for dependencies, options, and diagnostics.

Align two-arm state

This example uses left ArmState as baseline and matches right ArmState. Matching defaults to AtOrBefore with a fixed 7.5 ms right-arm tolerance; the first valid sample can be matched.

C++

```cpp
void aligned_arms(north::NorthSDK& sdk) {
    north::AlignmentSource left;
    left.kind = north::AlignmentDataKind::kArmState;
    left.side = north::Side::kLeft;
    auto right = left;
    right.side = north::Side::kRight;
    north::AlignmentOptions options;
    options.baseline = left;
    options.sources = {right};
    auto result = sdk.alignment().on_frame(options, [](const north::AlignedFrame& frame) {
        std::cout << frame.timestamp_ns << " complete=" << frame.complete << '\n';
    });
    if (!result) throw std::runtime_error(result.error().message);
    auto subscription = std::move(result.value());
    std::this_thread::sleep_for(std::chrono::seconds(5));
    subscription.close();
}
```

Python

```python
def aligned_arms(sdk):
    from north_sdk import AlignmentDataKind, AlignmentOptions, AlignmentSource, Side

    left = AlignmentSource()
    left.kind = AlignmentDataKind.ARM_STATE
    left.side = Side.LEFT
    right = AlignmentSource()
    right.kind = AlignmentDataKind.ARM_STATE
    right.side = Side.RIGHT
    options = AlignmentOptions()
    options.baseline = left
    options.sources = [right]
    subscription = sdk.alignment().on_frame(
        options, lambda frame: print(frame.timestamp_ns, frame.complete)
    )
    time.sleep(5)
    subscription.close()
```

`frame.samples[0]` is the right arm and `offset_ns` is right minus left time. See `AlignmentClient` for matching and diagnostics.

Query configuration instances

An empty `plugin_id` discovers all ready instances. Check both SDK failure and `ConfigResult`.

C++

```cpp
void config_instances(north::NorthSDK& sdk) {
    auto result = sdk.config().get();
    if (!result) throw std::runtime_error(result.error().message);
    if (result.value().result != north::ConfigResult::kOk)
        throw std::runtime_error("configuration query rejected");
    for (const auto& instance : result.value().instances)
        std::cout << instance.plugin_id << '\n';
}
```

Python

```python
def config_instances(sdk):
    from north_sdk import ConfigResult

    response = sdk.config().get()
    if response.result != ConfigResult.OK:
        raise RuntimeError(response.diagnostics)
    for instance in response.instances:
        print(instance.plugin_id)
```

Pass an ID returned by discovery, such as `get("chassis")`, for one instance. See `ConfigClient` for persistent SET/RESET parameters and timeout handling.

Master mode service

This example changes robot mode. Call it only when the application has confirmed that switching is allowed. The application selects requested. Master responses use result rather than success; continue observing state after success.

`requested` is an `OperationMode`: Teleoperation, Inference or Automation. Select a mode that corresponds to the CommandChannel used by later motion. See `MasterClient::set_mode` for parameters, results, and timeout handling.

C++

```cpp
void change_master_mode(north::NorthSDK& sdk, north::OperationMode requested) {
    auto result = sdk.master().set_mode(requested);
    if (!result) throw std::runtime_error("SDK error: " + result.error().message);
    if (result.value().result != north::MasterResult::kOk)
        throw std::runtime_error("Service rejected: " + result.value().message);
    std::cout << result.value().message << '\n';
}
```

Python

```python
def change_master_mode(sdk, requested):
    response = sdk.master().set_mode(requested)
    if response.result != MasterResult.OK:
        raise RuntimeError("Service rejected: " + response.message)
    print(response.message)
```

Send a single-arm target

First follow Prepare and end a task to configure the channel, devices and task. Generate seven finite rad targets in device order from fresh feedback within model limits, and use actual position feedback to confirm completion. See `ArmClient::send_joint_positions` for complete constraints.

C++

```cpp
void send_right_arm(north::NorthSDK& sdk, const std::array<float, 7>& target) {
    auto sent = sdk.arm(north::Side::kRight).send_joint_positions(target);
    if (!sent) throw std::runtime_error(sent.message());
}
```

Python

```python
def send_right_arm(sdk, target):
    if len(target) != 7:
        raise ValueError("Expected 7 joint positions")
    sdk.arm(Side.RIGHT).send_joint_positions(target)
```

No send error means local submission succeeded. Compare fresh right-arm feedback with the target using application-defined tolerance. Stop sending and reassess runtime conditions on kStaleState or kMotionNotAllowed.

WholeBody two-arm command

Use the same motion prerequisites, with both arms prepared. Pass an explicit task-compatible channel and application-planned targets. This example updates both arms only; inspect each arm feedback to determine completion.

C++

```cpp
void send_two_arms(north::NorthSDK& sdk, north::CommandChannel channel,
                   const std::array<float, 7>& left,
                   const std::array<float, 7>& right) {
    north::ArmCommand left_command, right_command;
    left_command.joints.position.assign(left.begin(), left.end());
    right_command.joints.position.assign(right.begin(), right.end());
    north::WholeBodyCommand command;
    command.left_arm = left_command;
    command.right_arm = right_command;
    auto sent = sdk.whole_body().send(channel, command);
    if (!sent) throw std::runtime_error(sent.message());
}
```

Python

```python
def send_two_arms(sdk, channel, left, right):
    if len(left) != 7 or len(right) != 7:
        raise ValueError("Expected 7 positions for each arm")
    left_command, right_command = ArmCommand(), ArmCommand()
    from north_sdk import JointCommand
    left_joints, right_joints = JointCommand(), JointCommand()
    left_joints.position = list(left)
    right_joints.position = list(right)
    left_command.joints = left_joints
    right_command.joints = right_joints
    command = WholeBodyCommand()
    command.left_arm = left_command
    command.right_arm = right_command
    sdk.whole_body().send(channel, command)
```